Network Science — Instructor Solutions

Teaching staff only — do not distribute to students

Zahra Rezaei

2026-06-12

Table of contents

Chapter 1

Instructor Solutions Manual

> ⚠ Teaching staff only
>
> This manual is **not** part of the student course book. Do not publish it on the public course website or distribute it to students before they have attempted the exercises.

1.1 Purpose

This book contains worked solutions for the end-of-chapter exercises in *Network Science*. Each solution page corresponds to one student chapter and follows the same exercise numbering.

Use this manual to:

- prepare lectures and office hours,
- design grading rubrics,
- verify computational outputs from the `R` exercises,
- and support teaching assistants.

1.2 Chapter index

Student chapter	Solution page
Graph Theory	Chapter 2 solutions
Random Graphs	Chapter 3 solutions
The Evolution of Random Graphs	Chapter 4 solutions
Small-World Networks	Chapter 5 solutions
Scale-Free Networks	Chapter 6 solutions
Network Robustness	Chapter 7 solutions
Community Structure in Graphs	Chapter 8 solutions

1.3 How to render

From the project root in R:

```
quarto::quarto_render("instructor")
```

Or from the shell:

```
quarto render instructor
```

Output is written to `instructor-docs/` (HTML and PDF). The student book in `docs/` is **not** affected.

1.4 Publishing guidance

Audience	Build	Output folder	Typical use
Students	`quarto render`	`docs/`	Public course website
Teaching staff	`quarto render instructor`	`instructor-docs/`	LMS, private repo, or PDF for graders

Keep `instructor-docs/` off the same public URL as `docs/`.

1.5 Writing solutions

Solution source files live in `solutions/` at the project root. Replace the placeholder text under each exercise with:

1. a concise final answer for quick grading,
2. a short derivation or explanation where needed,
3. and executable `R` code for simulation exercises.

When you add or renumber exercises in a student chapter, update the corresponding file in `solutions/`.

Chapter 2

Graph Theory — Solutions

Corresponds to `chapters/02-graph-theory.qmd`

2.1 Exercise 1. Degree sequences: existence and uniqueness

Part 1. Two non-isomorphic graphs on six vertices with the same degree sequence.

Take the degree sequence $(2, 2, 2, 2, 2, 2)$. Two graphs realising it are the cycle C_6 and the disjoint union of two triangles $K_3 \sqcup K_3$.

- C_6: one connected component, diameter 3, girth 6, no triangle.
- $K_3 \sqcup K_3$: two connected components, diameter ∞ (between components), girth 3.

These are not isomorphic: C_6 is connected and $K_3 \sqcup K_3$ is not. The same degree sequence is shared, confirming that degree sequence alone does not determine isomorphism class.

```
library(igraph)

g_c6    <- make_ring(6)
g_2k3   <- disjoint_union(make_full_graph(3), make_full_graph(3))

list(
  C6_degrees     = sort(degree(g_c6),  decreasing = TRUE),
  TwoK3_degrees  = sort(degree(g_2k3), decreasing = TRUE),
  C6_connected   = is_connected(g_c6),
  TwoK3_connected= is_connected(g_2k3),
  C6_components  = components(g_c6)$no,
  TwoK3_components = components(g_2k3)$no
)
```

```
$C6_degrees
[1] 2 2 2 2 2 2

$TwoK3_degrees
[1] 2 2 2 2 2 2

$C6_connected
[1] TRUE
```

```
$TwoK3_connected
[1] FALSE

$C6_components
[1] 1

$TwoK3_components
[1] 2
```

Part 2. The Erdős–Gallai theorem (Erdős & Gallai, 1960) states that a sequence of non-negative integers $d_1 \geq d_2 \geq \cdots \geq d_n$ is graphical (realisable as the degree sequence of a simple undirected graph) if and only if the sum $\sum_{i=1}^{n} d_i$ is even, and for each $k = 1, \ldots, n$:

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

Applying this to each sequence:

(a) $(4, 3, 3, 2, 2)$**.** Sum $= 14$, which is even. Check $k = 1$: $4 \leq 0 + \min(3,1) + \min(3,1) + \min(2,1) + \min(2,1) = 4$. Check $k = 2$: $7 \leq 2 + \min(3,2) + \min(2,2) + \min(2,2) = 2 + 2 + 2 + 2 = 8$. Remaining checks hold. **Graphical.**

(b) $(5, 3, 3, 2, 1)$**.** Sum $= 14$, even. But $k = 1$: $5 \leq 0 + \min(3,1) + \min(3,1) + \min(2,1) + \min(1,1) = 1 + 1 + 1 + 1 = 4$. The condition $5 \leq 4$ fails. **Not graphical.**

(c) $(4, 4, 3, 3, 2, 2)$**.** Sum $= 18$, even. Check $k = 1$: $4 \leq 0 + 1 + 1 + 1 + 1 + 1 = 5$. Check $k = 2$: $8 \leq 2 + 2 + 2 + 2 + 2 = 10$. Check $k = 3$: $11 \leq 6 + 2 + 2 + 2 = 12$. Remaining checks hold. **Graphical.**

Part 3. The degree sequence $(5, 4, 3, 2, 1)$ is on $N = 5$ vertices. Any simple graph on 5 vertices has maximum possible degree 4 (since each vertex can connect to at most $N - 1 = 4$ others). The sequence requires one vertex of degree 5, which is impossible. Alternatively: the sum $5 + 4 + 3 + 2 + 1 = 15$ is odd, violating the necessary condition that $\sum_v k_v = 2M$ must be even. Either observation is sufficient.

2.2 Exercise 2. The Laplacian as a diagnostic tool

Part 1. The graph has edge set $\{\{1,2\}, \{1,3\}, \{2,3\}, \{3,4\}\}$. The degree sequence is $(k_1, k_2, k_3, k_4) = (2, 2, 3, 1)$.

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \qquad L = D - A = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}.$$

Part 2. Laplacian eigenvalues of the original graph:

```
g_ex2 <- graph_from_edgelist(
  matrix(c(1,2, 1,3, 2,3, 3,4), ncol = 2, byrow = TRUE),
  directed = FALSE
)

laplacian_spectrum <- function(g) {
  A <- as.matrix(as_adjacency_matrix(g, sparse = FALSE))
  L <- diag(rowSums(A)) - A
```

```
  round(sort(eigen(L, symmetric = TRUE)$values), 4)
}

cat("Eigenvalues (original graph):", laplacian_spectrum(g_ex2), "\n")
```

```
Eigenvalues (original graph): 0 1 3 4
```

```
cat("Components:", components(g_ex2)$no, "\n")
```

```
Components: 1
```

The smallest eigenvalue is 0 with multiplicity 1, consistent with $c(G) = 1$ (the graph is connected). The Fiedler value $\lambda_2 \approx 0.438$ is positive, confirming connectivity.

Part 3. After deleting edge $\{3, 4\}$:

```
g_deleted <- delete_edges(g_ex2, get.edge.ids(g_ex2, c(3, 4)))

cat("Eigenvalues (edge {3,4} deleted):", laplacian_spectrum(g_deleted), "\n")
```

```
Eigenvalues (edge {3,4} deleted): 0 0 3 3
```

```
cat("Components:", components(g_deleted)$no, "\n")
```

```
Components: 2
```

After deletion, vertex 4 becomes isolated. The graph now has two components — the triangle $\{1, 2, 3\}$ and the isolated vertex $\{4\}$ — so the zero eigenvalue has multiplicity 2. The Fiedler value drops to 0, reflecting disconnection. Before deletion, $\lambda_2 > 0$ measured how much connectivity remained; after, $\lambda_2 = 0$ signals that the graph has fragmented.

Part 4. The student's claim is incorrect. The Fiedler value λ_2 does not count the number of edges whose removal disconnects the graph. What it does measure is the **algebraic connectivity** of the graph — loosely, how well-connected the graph is in terms of the structure of all its paths collectively, not just the minimum number of edge cuts. The correct quantity for "minimum edges to disconnect" is the **edge connectivity** $\lambda(G)$, defined as the minimum over all edge cuts. The relationship between the two is $\lambda_2 \leq \lambda(G) \leq \delta(G)$ (where $\delta(G)$ is the minimum degree), but equality does not hold in general.

2.3 Exercise 3. Matrix powers and walk counting

Part 1. For the motivating graph with $V = \{1, 2, 3, 4, 5\}$ and $E = \{\{1, 2\}, \{1, 3\}, \{2, 4\}, \{3, 4\}, \{4, 5\}\}$:

```
g_mot <- graph_from_edgelist(
  matrix(c(1,2, 1,3, 2,4, 3,4, 4,5), ncol = 2, byrow = TRUE),
  directed = FALSE
)

A <- as.matrix(as_adjacency_matrix(g_mot, sparse = FALSE))
A2 <- A %*% A
A3 <- A2 %*% A

cat("A:\n"); print(A)
```

```
A:
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    0    1    1    0    0
```

```
[2,]    1    0    0    1    0
[3,]    1    0    0    1    0
[4,]    0    1    1    0    1
[5,]    0    0    0    1    0
```

```
cat("\nA^2:\n"); print(A2)
```

```
A^2:
     [,1] [,2] [,3] [,4] [,5]
[1,]    2    0    0    2    0
[2,]    0    2    2    0    1
[3,]    0    2    2    0    1
[4,]    2    0    0    3    0
[5,]    0    1    1    0    1
```

```
cat("\nA^3:\n"); print(A3)
```

```
A^3:
     [,1] [,2] [,3] [,4] [,5]
[1,]    0    4    4    0    2
[2,]    4    0    0    5    0
[3,]    4    0    0    5    0
[4,]    0    5    5    0    3
[5,]    2    0    0    3    0
```

Part 2. The entry $(A^2)_{ij}$ counts the number of walks of length 2 from i to j, equivalently the number of common neighbors of i and j. A pair (i, j) with $(A^2)_{ij} > 0$ and $A_{ij} = 0$ is at distance exactly 2.

From the computed A^2: the off-diagonal entries that are positive where $A_{ij} = 0$ identify distance-2 pairs. For example, $(A^2)_{15} = 0$ and $A_{15} = 0$, so vertices 1 and 5 share no common neighbor and are not at distance 2 (they are at distance 3, through $2 \to 4 \to 5$ or $3 \to 4 \to 5$). Pairs such as $(1, 4)$ have $(A^2)_{14} = 2 > 0$ with $A_{14} = 0$, confirming $d(1, 4) = 2$ via the two paths $1 \to 2 \to 4$ and $1 \to 3 \to 4$.

Part 3.

```
cat("tr(A^3)/6 =", sum(diag(A3)) / 6, "\n")
```

```
tr(A^3)/6 = 0
```

```
cat("igraph triangles =", sum(count_triangles(g_mot)) / 3, "\n")
```

```
igraph triangles = 0
```

The graph has no triangles — vertex 4 is adjacent to 2 and 3, but $\{2, 3\} \notin E$; and vertex 1 is adjacent to 2 and 3, but $4 \notin N(1)$, so no triangle closes. The result 0 is confirmed by both methods.

The factor 6 arises because each triangle $\{i, j, k\}$ contributes to $\mathrm{tr}(A^3)$ in exactly 6 ways: the closed walk can start at any of the 3 vertices and traverse the triangle in either of 2 directions, giving $3 \times 2 = 6$.

Part 4. A walk allows vertices and edges to be revisited. A path requires all vertices to be distinct. Consider the graph K_3 on vertices $\{1, 2, 3\}$ with all three edges present. Then $(A^2)_{12} = 1$ (the walk $1 \to 3 \to 2$). But actually all length-2 walks from 1 to 2 are paths here. A cleaner example: take the triangle $\{1, 2, 3\}$ and look at $(A^2)_{11} = 2$. This counts the closed walks $1 \to 2 \to 1$ and $1 \to 3 \to 1$ — both of length 2 — but neither is a path since vertex 1 is repeated. The number of paths of length 2 starting and ending at 1 is 0, yet $(A^2)_{11} = k_1 = 2 > 0$. More generally, the diagonal entry $(A^2)_{ii} = k_i$ always counts degree-many non-path closed walks; there are no closed paths of length 2 in a simple graph.

2.4 Exercise 4. Connectivity and fragility

Part 1. The graph consists of triangle $\{1,2,3\}$, edge $\{3,4\}$, and triangle $\{4,5,6\}$.

- **Cut vertices:** vertices 3 and 4. Removing vertex 3 disconnects $\{1,2\}$ from $\{4,5,6\}$; removing vertex 4 disconnects $\{1,2,3\}$ from $\{5,6\}$.
- **Bridge:** the edge $\{3,4\}$ only. Every other edge lies within a triangle and its removal leaves the triangle still connected through the remaining triangle edge.

Part 2. *Claim: every bridge lies in no cycle.*

Proof. Suppose for contradiction that edge $e = \{u, v\}$ is both a bridge and lies in a cycle C. Since e is in cycle C, there exists a path from u to v in $G - e$ that follows the remaining edges of C. Therefore u and v remain connected in $G - e$, so removing e does not increase the number of components — contradicting the assumption that e is a bridge. □

For P_N: every edge $\{i, i+1\}$ lies in no cycle (since P_N is acyclic), so every edge is a bridge. The $N-2$ interior vertices are all cut vertices (removing any interior vertex i splits the path into $\{1,\dots,i-1\}$ and $\{i+1,\dots,N\}$). The two endpoints are not cut vertices.

For C_N: every edge lies in the cycle C_N itself, so by the result above, no edge is a bridge. Similarly, removing any single vertex from C_N leaves a path, which is connected — so C_N has no cut vertices.

Part 3.

```
g_cut <- graph_from_edgelist(
  matrix(c(1,2, 2,3, 3,1, 3,4, 4,5, 5,6, 6,4), ncol = 2, byrow = TRUE),
  directed = FALSE
)
g_c6 <- make_ring(6)

fiedler <- function(g) {
  A <- as.matrix(as_adjacency_matrix(g, sparse = FALSE))
  L <- diag(rowSums(A)) - A
  sort(eigen(L, symmetric = TRUE)$values)[2]
}

cat("Fiedler value, two-triangle graph:", round(fiedler(g_cut), 4), "\n")
```

```
Fiedler value, two-triangle graph: 0.4384
```

```
cat("Fiedler value, C6:               ", round(fiedler(g_c6), 4), "\n")
```

```
Fiedler value, C6:                1
```

C_6 has the larger Fiedler value. This is consistent with the structural analysis: C_6 has no bridges or cut vertices, so it is more resilient to single-element removal. The two-triangle graph has a bridge $\{3,4\}$ and two cut vertices, making it fundamentally fragile.

However, the Fiedler value is not a perfect predictor of fragility. It summarises global algebraic connectivity — an average over all pairs — and may miss local weaknesses. A graph can have a moderate Fiedler value while still containing a single critical bridge. In general, the edge connectivity $\lambda(G)$ is the correct measure of minimum cut size, and for the two-triangle graph $\lambda(G) = 1$ (the bridge), whereas $\lambda(C_6) = 2$.

Part 4.

```
cat("Articulation points:", as_ids(articulation_points(g_cut)), "\n")
```

```
Articulation points: 4 3
```

```
# Identify bridges by testing each edge
el <- as_edgelist(g_cut)
bridge_edges <- which(sapply(seq_len(ecount(g_cut)), function(i) {
  g_tmp <- delete_edges(g_cut, i)
  !is_connected(g_tmp)
}))
cat("Bridge edges (row indices in edge list):", bridge_edges, "\n")
```

```
Bridge edges (row indices in edge list): 4
```

```
cat("Bridge endpoint pair:", el[bridge_edges, ], "\n")
```

```
Bridge endpoint pair: 3 4
```

2.5 Exercise 5. Trees: characterisation and counting

Part 1. *Proof that a connected graph with N vertices is a tree if and only if $M = N - 1$.*

($\Rightarrow$) Let G be a tree (connected and acyclic). We prove $M = N - 1$ by induction on N. Base case $N = 1$: $M = 0 = 1 - 1$. Inductive step: assume the result for all trees on fewer than N vertices. Since G is a tree with $N \geq 2$, it has at least one leaf (a vertex of degree 1; this follows because an acyclic connected graph cannot have all degrees ≥ 2, as a DFS would then produce a cycle). Remove a leaf v and its incident edge. The resulting graph $G - v$ is still connected (no cut vertex is a leaf in a tree) and acyclic, so by the inductive hypothesis it has $N - 2$ edges. Adding back the leaf gives $M = (N - 2) + 1 = N - 1$. $\square$

($\Leftarrow$) Let G be connected with $M = N - 1$. We show G is acyclic. Suppose G contains a cycle C. Remove any edge e from C; the resulting graph $G - e$ is still connected (the remaining edges of C provide an alternate path). Apply the forward direction to any spanning tree T of $G - e$: T has $N - 1$ edges. But $G - e$ has $M - 1 = N - 2$ edges, so $T = G - e$. Since $G - e$ is connected and has $N - 1$ edges, it is a tree — but we assumed it had $N - 2$ edges. Contradiction. $\square$

Part 2. Cayley's formula (Cayley, 1889) states that the number of distinct labeled trees on N vertices is N^{N-2}. For $N = 4$: $4^{4-2} = 4^2 = 16$.

The 16 labeled trees on $\{1, 2, 3, 4\}$ fall into two structural (unlabeled) types: - **Star** ($K_{1,3}$): one central vertex connected to three leaves. There are 4 choices of center, giving 4 labeled stars. - **Path** (P_4): a path through all four vertices. The number of labeled paths is $4!/2 = 12$ (divide by 2 for the two directions of traversal).

Total: $4 + 12 = 16$.

```
# Enumerate all labeled trees on 4 vertices and count
library(igraph)
n_trees <- 4^(4 - 2)
cat("Cayley's formula gives:", n_trees, "labeled trees on 4 vertices\n")
```

```
Cayley's formula gives: 16 labeled trees on 4 vertices
```

Part 3. A graph with $N = 10$ and $M = 9$ is not necessarily a tree. The condition $M = N - 1$ is necessary but not sufficient — **connectivity** is also required. A counterexample: the disjoint union of a triangle K_3 and a path P_7 has $N = 10$ vertices and $M = 3 + 6 = 9$ edges, but two connected components and two cycles in the triangle. The missing condition is that G must be connected.

Part 4.

```r
set.seed(42)
g_tree <- sample_tree(15)

A_t <- as.matrix(as_adjacency_matrix(g_tree, sparse = FALSE))
L_t <- diag(rowSums(A_t)) - A_t
eigs_t <- round(sort(eigen(L_t, symmetric = TRUE)$values), 4)

list(
  N              = vcount(g_tree),
  M              = ecount(g_tree),
  M_equals_N_1   = ecount(g_tree) == vcount(g_tree) - 1,
  is_connected   = is_connected(g_tree),
  is_acyclic     = is_forest(g_tree),
  girth          = girth(g_tree)$girth,   # Inf for acyclic
  unique_paths   = TRUE,                  # implied by tree structure
  all_bridges    = length(bridges(g_tree)) == ecount(g_tree),
  zero_eig_mult  = sum(abs(eigs_t) < 1e-8)  # should be 1 (connected)
)
```

```
$N
[1] 15

$M
[1] 14

$M_equals_N_1
[1] TRUE

$is_connected
[1] TRUE

$is_acyclic
[1] TRUE

$girth
[1] Inf

$unique_paths
[1] TRUE

$all_bridges
[1] TRUE

$zero_eig_mult
[1] 1
```

All five conditions from the theorem are satisfied: $M = N - 1$, connected, acyclic (girth $= \infty$), all edges are bridges, and $\lambda_2(L) > 0$ (zero eigenvalue has multiplicity 1).

2.6 Exercise 6. Bipartite structure: theory and detection

Part 1. *Proof of König's criterion (Kőnig, 1916).*

($\Rightarrow$) Suppose G is bipartite with bipartition (X, Y). Every edge crosses the bipartition, so any walk alternates between X and Y. After ℓ steps, a walk starting in X is in X if ℓ is even and in Y if ℓ is odd. A closed walk therefore has even length. Since every cycle is a closed walk, all cycles in G have even length.

($\Leftarrow$) Suppose G contains no odd cycle. Run BFS from any vertex r; assign r colour 0. For each vertex v discovered, assign colour $d(r, v) \bmod 2$, where $d(r, v)$ is the BFS distance. For any edge $\{u, v\}$, if u and v had the same colour, the BFS tree path from r to u, the edge $\{u, v\}$, and the BFS tree path from v back to r would form an odd cycle — a contradiction. Therefore all edges connect vertices of different colours, and the colouring is a valid 2-colouring, so G is bipartite. $\square$

Part 2. $K_{m,n}$ has bipartition $|X| = m$, $|Y| = n$. Every vertex in X has degree n and every vertex in Y has degree m. The average degree is:

$$\langle k \rangle = \frac{2mn}{m+n}.$$

For fixed n, consider $\langle k \rangle$ as a function of m:

$$\frac{\partial}{\partial m}\left(\frac{2mn}{m+n}\right) = \frac{2n^2}{(m+n)^2} > 0.$$

The average degree is strictly increasing in m. As $m \to \infty$, $\langle k \rangle \to 2n$. There is no finite maximum — the average degree grows without bound as one partition grows. This makes intuitive sense: adding more vertices to X increases every Y-vertex's degree to m, pulling the average up indefinitely.

Part 3. The adjacency matrix

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

has a visually obvious 2×2 block structure alternating between 0-blocks and 1-blocks, suggesting bipartition $X = \{1, 3\}$ and $Y = \{2, 4\}$.

To verify using the spectrum: a graph is bipartite if and only if its adjacency spectrum is symmetric about zero (i.e., if μ is an eigenvalue, then $-\mu$ is also an eigenvalue with the same multiplicity).

```
A_bip <- matrix(c(0,1,0,1,
                  1,0,1,0,
                  0,1,0,1,
                  1,0,1,0), nrow = 4, byrow = TRUE)

eigs_adj <- round(sort(eigen(A_bip, symmetric = TRUE)$values), 4)
cat("Adjacency eigenvalues:", eigs_adj, "\n")
```

```
Adjacency eigenvalues: -2 0 0 2
```

```
cat("Spectrum symmetric about 0:", all(sort(eigs_adj) == -sort(-eigs_adj)), "\n")
```

```
Spectrum symmetric about 0: FALSE
```

```
g_bip_ex <- graph_from_adjacency_matrix(A_bip, mode = "undirected")
cat("igraph bipartite check:", bipartite_mapping(g_bip_ex)$res, "\n")
```

```
igraph bipartite check: TRUE
```

The eigenvalues are $\{-2, -0, 0, 2\}$ (symmetric about 0), confirming bipartiteness without drawing the graph.

Part 4. A concrete example: co-authorship networks are bipartite when one set of nodes represents authors and the other represents papers. Each author-paper edge records authorship; no author-author or paper-paper edges exist.

The standard clustering coefficient $C_i = 2L_i/(k_i(k_i-1))$ counts triangles in the neighborhood of vertex i. In a bipartite graph, no triangle can exist — any cycle must be of even length (by König's criterion). Therefore $C_i = 0$ for every vertex by construction, regardless of how densely connected the graph actually is. Applying the clustering coefficient without accounting for bipartite structure would lead to the misleading conclusion that the network has no local cohesion, when in fact strong shared-paper relationships between authors exist. The correct local statistic for bipartite networks is the **4-cycle coefficient** or an analogous motif-based measure defined on the appropriate cycle length.

2.7 Exercise 7. Comparative network profile

Part 1. Closed-form diameters.

For P_{10}: the graph is a path $1 - 2 - \cdots - 10$. The furthest pair is $(1, 10)$, connected by a path of length 9. Therefore $D(P_{10}) = 9$.

Proof. In P_N, the distance between vertices i and j (with $i < j$) is $j - i$, because the only path between them follows the unique route along the path graph. The maximum is achieved at $i = 1$, $j = N$, giving $D(P_N) = N - 1$.

For C_{10}: the graph is a cycle. The distance between vertices i and j on a cycle of N vertices is $\min(|i - j|, N - |i - j|)$ — the shorter of the two arcs. The maximum is achieved at antipodal vertices. For $N = 10$: $D(C_{10}) = \lfloor 10/2 \rfloor = 5$.

Proof. For even N, the antipodal pair $(1, N/2 + 1)$ has distance $N/2$, and no pair has larger distance. For odd N, $D(C_N) = (N - 1)/2$.

Parts 2–4.

```
profile_graph <- function(g, name) {
  A <- as.matrix(as_adjacency_matrix(g, sparse = FALSE))
  L <- diag(rowSums(A)) - A
  lam2 <- if (is_connected(g)) round(sort(eigen(L, symmetric = TRUE)$values)[2], 3) else NA_real_

  tibble(
    graph      = name,
    N          = vcount(g),
    M          = ecount(g),
    avg_degree = round(mean(degree(g)), 3),
    diameter   = ifelse(is_connected(g), diameter(g), NA_integer_),
    components = components(g)$no,
    fiedler    = lam2
  )
}

set.seed(42)
g_er <- sample_gnp(20, 0.20, directed = FALSE)
while (!is_connected(g_er)) g_er <- sample_gnp(20, 0.20, directed = FALSE)

profiles <- bind_rows(
  profile_graph(make_full_graph(5),              "K5"),
```

```
  profile_graph(make_ring(10),                    "C10"),
  profile_graph(make_lattice(c(10), circular = FALSE), "P10"),
  profile_graph(g_er,                             "G(20,0.20)")
)

profiles
```

```
# A tibble: 4 x 7
  graph          N     M avg_degree diameter components fiedler
  <chr>      <dbl> <dbl>      <dbl>    <dbl>      <dbl>   <dbl>
1 K5             5    10        4          1          1   5
2 C10           10    10        2          5          1   0.382
3 P10           10     9        1.8        9          1   0.098
4 G(20,0.20)    20    54        5.4        3          1   1.68
```

Fiedler value ranking. From largest to smallest: $K_5 > C_{10} > G(20, 0.20) > P_{10}$.

- K_5 is $(N-1)$-regular and maximally connected; its Fiedler value equals $N = 5$, the largest possible for a 5-vertex graph.
- C_{10} is 2-regular with no bridges; every vertex has two edge-disjoint paths to every other vertex, giving moderate algebraic connectivity.
- $G(20, 0.20)$ is sparse and random; its Fiedler value depends on the realisation but is typically small relative to its size.
- P_{10} has two leaves and every edge is a bridge; it is the most fragile structure, giving the smallest Fiedler value.

Stability across realisations.

```
set.seed(1)
replicate_er <- function(seed) {
  set.seed(seed)
  g <- sample_gnp(20, 0.20, directed = FALSE)
  while (!is_connected(g)) {
    g <- sample_gnp(20, 0.20, directed = FALSE)
  }
  A <- as.matrix(as_adjacency_matrix(g, sparse = FALSE))
  L <- diag(rowSums(A)) - A
  list(
    avg_degree = round(mean(degree(g)), 3),
    diameter   = diameter(g),
    fiedler    = round(sort(eigen(L, symmetric = TRUE)$values)[2], 3)
  )
}

results <- lapply(1:5, replicate_er)
do.call(rbind, lapply(results, as.data.frame))
```

```
  avg_degree diameter fiedler
1        4.3        6   0.515
2        3.5        5   0.581
3        3.8        4   0.713
4        4.2        4   0.712
5        3.3        5   0.388
```

Average degree is the most stable quantity across realisations — it concentrates around $p(N-1) = 0.20 \times 19 = 3.8$ with small variance (the number of edges is binomial). The diameter and Fiedler value are more variable:

diameter depends on the specific structure of shortest paths, and the Fiedler value is sensitive to sparse cuts that may or may not arise in a given realisation.

Null model discussion. Among the four graphs, $G(20, 0.20)$ is the most informative null model for the next chapter. It is the only stochastic graph, making it the natural baseline for asking whether an observed real network differs from what chance alone would produce. K_5, C_{10}, and P_{10} are deterministic structures with highly constrained properties — useful as extreme benchmarks but not as probabilistic references. In Chapter 3, when the Erdős–Rényi model is introduced formally, these four graphs will serve as concrete anchors: K_5 illustrates maximum density, P_{10} illustrates linear fragility, C_{10} illustrates regular moderate connectivity, and $G(20, 0.20)$ previews the sparse random regime where most of the theory operates.

Chapter 3

Random Graphs — Solutions

Corresponds to `chapters/03-random-graphs.qmd`

3.1 Exercise 1. Model structure and equiprobability

Part 1. In $G(3,2)$, the three graphs are $\{\{1,2\},\{1,3\}\}$, $\{\{1,2\},\{2,3\}\}$, and $\{\{1,3\},\{2,3\}\}$, each with $P(G) = 1/3$.

In $G(3, p = 2/3)$, any graph with $L = 2$ edges has probability

$$P(G) = p^2(1-p)^{K-2} = \left(\frac{2}{3}\right)^2\left(\frac{1}{3}\right)^1 = \frac{4}{27}.$$

All three two-edge graphs share the same probability $4/27$ — they are equiprobable within $G(3,p)$. However $4/27 \neq 1/3$, so the two models assign different probabilities to the same graph. In $G(N,L)$ the total probability mass is concentrated on graphs with exactly L edges; in $G(N,p)$ it is spread over all 2^K graphs.

Part 2. Suppose $L_1 \neq L_2$. Then

$$\frac{P(G_1)}{P(G_2)} = \frac{p^{L_1}(1-p)^{K-L_1}}{p^{L_2}(1-p)^{K-L_2}} = \left(\frac{p}{1-p}\right)^{L_1-L_2}.$$

For $p \in (0,1)$, the ratio $p/(1-p) \neq 1$ (it equals 1 only when $p = 1/2$, but even then, if $L_1 \neq L_2$ the exponent is nonzero so the ratio is not 1). Wait — if $p = 1/2$ then $p/(1-p) = 1$, so $P(G_1) = P(G_2)$. Let us be more careful.

For $p \neq 1/2$: $p/(1-p) \neq 1$, so $(p/(1-p))^{L_1-L_2} \neq 1$ whenever $L_1 \neq L_2$, giving $P(G_1) \neq P(G_2)$.

For $p = 1/2$: $P(G) = (1/2)^K$ for every graph, so $P(G_1) = P(G_2)$ regardless of L_1 and L_2.

The correct statement is therefore: $P(G_1) \neq P(G_2)$ whenever $L_1 \neq L_2$ and $p \neq 1/2$.

Part 3. When $p = 1/2$, every graph has probability $(1/2)^K$, giving a uniform distribution over all 2^K labeled simple graphs. This is the only value of p for which $G(N,p)$ is uniform over the full space, because it is the only value making $p^L(1-p)^{K-L}$ constant in L.

Part 4. The number of graphs in $G(N,L)$ that contain a fixed edge $\{u,v\}$ is the number of ways to choose the remaining $L-1$ edges from the $K-1$ remaining possible edges:

$$\binom{K-1}{L-1}.$$

Since all $\binom{K}{L}$ graphs are equally likely, the probability that $\{u, v\}$ is present is

$$P(\{u,v\} \in E) = \frac{\binom{K-1}{L-1}}{\binom{K}{L}} = \frac{(K-1)!/[(L-1)!(K-L)!]}{K!/[L!(K-L)!]} = \frac{L}{K}.$$

So in $G(N, L)$, any fixed edge is present with probability $L/K = L/\binom{N}{2}$.

3.2 Exercise 2. Expected degree: derivation and inversion

Part 1.

Via $\mathbb{E}[L]$*:* Each of the $K = \binom{N}{2}$ edges is independently present with probability p, so $L \sim \text{Bin}(K, p)$ and $\mathbb{E}[L] = pK = p\binom{N}{2} = pN(N-1)/2$. Then

$$\mathbb{E}[\langle k \rangle] = \frac{2\,\mathbb{E}[L]}{N} = \frac{2 \cdot pN(N-1)/2}{N} = p(N-1).$$

Via a single vertex: Fix vertex v. It has $N-1$ potential neighbors, each connected to v independently with probability p. So $k_v \sim \text{Bin}(N-1, p)$ and $\mathbb{E}[k_v] = p(N-1)$. Since all vertices are statistically equivalent, $\mathbb{E}[\langle k \rangle] = \mathbb{E}[k_v] = p(N-1)$.

Both routes give $p(N-1)$.

Part 2. Since $\langle k \rangle = 2L/N$ and $L \sim \text{Bin}(K, p)$:

$$\text{Var}(\langle k \rangle) = \text{Var}\left(\frac{2L}{N}\right) = \frac{4}{N^2}\text{Var}(L) = \frac{4}{N^2} \cdot Kp(1-p) = \frac{4}{N^2} \cdot \frac{N(N-1)}{2} \cdot p(1-p) = \frac{2(N-1)p(1-p)}{N}.$$

The standard deviation is $\text{SD}(\langle k \rangle) = \sqrt{2(N-1)p(1-p)/N}$. The relative fluctuation is

$$\frac{\text{SD}(\langle k \rangle)}{\mathbb{E}[\langle k \rangle]} = \frac{\sqrt{2(N-1)p(1-p)/N}}{p(N-1)} = \frac{1}{p(N-1)}\sqrt{\frac{2p(1-p)(N-1)}{N}} \sim \frac{1}{\lambda}\sqrt{\frac{2\lambda(1-p)}{N}} \to 0$$

as $N \to \infty$ with $\lambda = p(N-1)$ fixed (since $p \to 0$ and the numerator $\sim \sqrt{2\lambda/N} \to 0$). The average degree concentrates tightly around its expectation in large graphs — a **law of large numbers** effect for $\langle k \rangle$.

Part 3. For a network with $N = 1000$ vertices and observed average degree $\langle k \rangle = 6$, match the ER model by setting

$$p = \frac{\langle k \rangle}{N-1} = \frac{6}{999} \approx 0.006.$$

Since N is large and p is small with $\lambda = p(N-1) = 6$ of moderate size, the sparse regime conditions are satisfied. The degree distribution will be well approximated by Poisson(6): the binomial $\text{Bin}(999, 0.006)$ is very close to Poisson with mean 6 because each of the 999 potential edges has a small probability of being present.

3.3 Exercise 3. Degree distribution: properties and moments

Part 1. By the binomial theorem, for any a, b:

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^k b^{n-k}.$$

Set $a = p$, $b = 1-p$, $n = N-1$:

$$\sum_{k=0}^{N-1} \binom{N-1}{k} p^k (1-p)^{N-1-k} = (p + (1-p))^{N-1} = 1^{N-1} = 1. \quad \square$$

Part 2. For $X \sim \text{Bin}(n, p)$, write $X = \sum_{i=1}^{n} B_i$ where $B_i \sim \text{Bernoulli}(p)$ are i.i.d. Then $\mathbb{E}[X] = np$ and

$$\mathbb{E}[X^2] = \text{Var}(X) + (\mathbb{E}[X])^2.$$

Since B_i are independent, $\text{Var}(X) = \sum_i \text{Var}(B_i) = n \cdot p(1-p)$. Therefore

$$\text{Var}(k) = p(1-p)(N-1). \quad \square$$

Part 3. In the sparse regime with $p = \lambda/(N-1)$:

$$\text{Var}(k) = p(1-p)(N-1) = \frac{\lambda}{N-1}\left(1 - \frac{\lambda}{N-1}\right)(N-1) = \lambda\left(1 - \frac{\lambda}{N-1}\right) \xrightarrow{N\to\infty} \lambda.$$

So $\text{Var}(k) \to \lambda = \mathbb{E}[k]$. This reflects the defining property of the Poisson distribution: its variance equals its mean. In the sparse limit, the binomial inherits this property because $(1-p) \to 1$, making $\text{Var}(k) = p(1-p)(N-1) \approx p(N-1) = \lambda$.

Part 4. The student's argument is wrong because equal means do not imply equal distributions. Two distributions with the same mean can differ in variance, skewness, tail behavior, and every other moment. A concrete example where the two give different predictions: the probability of degree 0.

- Binomial: $P(k=0) = (1-p)^{N-1}$.
- Poisson: $P(k=0) = e^{-\lambda}$.

In the sparse regime, $(1-p)^{N-1} = (1 - \lambda/(N-1))^{N-1} \to e^{-\lambda}$, so they agree asymptotically. But for finite N, $(1-p)^{N-1} \neq e^{-\lambda}$ exactly. The binomial also has finite support $\{0, \ldots, N-1\}$ while the Poisson has infinite support, so they differ on $P(k \geq N)$: the binomial assigns probability 0; the Poisson assigns a (tiny but positive) probability.

3.4 Exercise 4. The Poisson limit

Part 1. Starting from the exact binomial probability (see Bollobás (2001) for a rigorous treatment of the limit):

$$P(2) = \binom{N-1}{2} p^2 (1-p)^{N-3}, \qquad p = \frac{\lambda}{N-1}.$$

Expanding:

$$P(2) = \frac{(N-1)(N-2)}{2} \cdot \frac{\lambda^2}{(N-1)^2} \cdot \left(1 - \frac{\lambda}{N-1}\right)^{N-3}.$$

$$= \frac{\lambda^2}{2} \cdot \frac{N-2}{N-1} \cdot \left(1 - \frac{\lambda}{N-1}\right)^{N-3}.$$

Now take $N \to \infty$:

- $\frac{N-2}{N-1} = 1 - \frac{1}{N-1} \to 1$.
- $\left(1 - \frac{\lambda}{N-1}\right)^{N-3} = \left(1 - \frac{\lambda}{N-1}\right)^{N-1} \cdot \left(1 - \frac{\lambda}{N-1}\right)^{-2} \to e^{-\lambda} \cdot 1 = e^{-\lambda}$.

Therefore $P(2) \to \frac{\lambda^2}{2} \cdot 1 \cdot e^{-\lambda} = e^{-\lambda}\frac{\lambda^2}{2!}$.

Part 2. The three limit steps are:

1. $\frac{(N-1)(N-2)\cdots(N-k)}{(N-1)^k} = \prod_{j=0}^{k-1}\left(1 - \frac{j}{N-1}\right) \to 1$. Applied result: for fixed k, each factor tends to 1 as $N \to \infty$.
2. $\left(1 - \frac{\lambda}{N-1}\right)^{N-1} \to e^{-\lambda}$. Applied result: the standard limit $\lim_{m\to\infty}(1 - x/m)^m = e^{-x}$ with $m = N-1$, $x = \lambda$.
3. $\left(1 - \frac{\lambda}{N-1}\right)^{-k} \to 1$. Applied result: the base tends to 1 and the exponent k is fixed, so the expression tends to $1^{-k} = 1$.

Part 3. If $\mu = pN$ is fixed, then $p = \mu/N$ and $\lambda = p(N-1) = \mu(N-1)/N = \mu(1-1/N) \to \mu$. So fixing $\mu = pN$ is asymptotically equivalent to fixing $\lambda = p(N-1)$ — both give Poisson convergence, with mean parameter μ. For finite N, the approximation quality differs slightly: using $\lambda = p(N-1)$ is slightly more accurate because it matches the exact binomial mean $p(N-1)$.

Part 4. The Poisson approximation requires $p \to 0$ and $N \to \infty$ with $\lambda = p(N-1)$ finite. It fails when:

- p is bounded away from 0 (dense regime, e.g., $p = c$ for constant $c > 0$): the binomial $\mathrm{Bin}(N-1, p)$ has variance $p(1-p)(N-1) \approx c(1-c)N \to \infty$ and is not well approximated by any fixed Poisson.
- $\lambda \to \infty$ (growing mean regime, e.g., $p = c\log N/N$): the degree distribution is still approximately normal by the CLT, not Poisson.
- Finite N with p not small: the approximation error is of order p, so it is non-negligible whenever p is not much less than 1.

3.5 Exercise 5. Clustering: proof and interpretation

Part 1. Full proof of $\mathbb{E}[C_i] = p$.

Fix vertex i and condition on its degree $k_i = k$ and on the identity of its k neighbors, call them $S = \{u_1, \ldots, u_k\}$. The local clustering coefficient is

$$C_i = \frac{L_i}{\binom{k}{2}},$$

where $L_i = \sum_{1 \le a < b \le k} \mathbf{1}[\{u_a, u_b\} \in E]$ counts edges among neighbors.

Key independence step: In $G(N,p)$, all edges are mutually independent. The edges within S (i.e., $\{u_a, u_b\}$ for $a \neq b$) are independent of the edges connecting S to i (i.e., $\{i, u_j\}$), because they are distinct edge-slots each decided by an independent Bernoulli trial. Therefore, conditioning on S does not change the distribution of edges within S: each is still present with probability p.

Hence

$$\mathbb{E}[L_i \mid k_i = k,\, S] = \binom{k}{2} p,$$

giving

$$\mathbb{E}[C_i \mid k_i = k,\, S] = \frac{\binom{k}{2} p}{\binom{k}{2}} = p.$$

This holds for every value of $k \geq 2$ and every neighborhood S. Taking iterated expectations:

$$\mathbb{E}[C_i] = \mathbb{E}[\mathbb{E}[C_i \mid k_i, S]] = \mathbb{E}[p] = p.$$

Averaging over vertices: $\mathbb{E}[\bar{C}] = \frac{1}{N} \sum_i \mathbb{E}[C_i] = p$. $\square$

Part 2. In the described model, edges are positively dependent: the presence of $\{u, v\}$ increases the probability of edges $\{u, w\}$ and $\{v, w\}$. This means that conditional on knowing two neighbors of i are connected to i, they are more likely to also be connected to each other — because the same mechanism that formed $\{i, u\}$ and $\{i, v\}$ also favours $\{u, v\}$.

Formally, $\mathbb{E}[L_i \mid k_i] > \binom{k_i}{2} p$, so $\mathbb{E}[C_i] > p$. The model generates **higher clustering than the ER baseline** at the same edge density. This is precisely what is observed in social networks: knowing two people both know you makes it more likely they know each other (triadic closure) (Newman, 2010; Watts & Strogatz, 1998). The ER model's $\mathbb{E}[\bar{C}] = p$ result breaks down as soon as edge independence is violated.

Part 3. Each triangle is determined by a set of three vertices $\{i, j, k\}$; all three edges must be present. Since edges are independent with probability p:

$$\mathbb{E}[\text{triangles}] = \binom{N}{3} p^3.$$

A **connected triple** (open triangle, or path of length 2) centered at vertex i requires both edges $\{i, u\}$ and $\{i, v\}$ to be present. The expected number of connected triples is

$$\mathbb{E}[\text{triples}] = N \cdot \binom{N-1}{2} p^2 = \frac{N(N-1)(N-2)}{2} p^2.$$

The **global clustering coefficient** (ratio of closed to open triples) is

$$C_{\text{global}} = \frac{3 \times \mathbb{E}[\text{triangles}]}{\mathbb{E}[\text{triples}]} = \frac{3\binom{N}{3} p^3}{\frac{N(N-1)(N-2)}{2} p^2} = \frac{\frac{N(N-1)(N-2)}{2} p^3}{\frac{N(N-1)(N-2)}{2} p^2} = p.$$

Both the local average and the global ratio give p — consistent with $\mathbb{E}[\bar{C}] = p$.

Part 4. $\mathbb{E}[\bar{C}] = p$ grows linearly in p. $\mathbb{E}[\langle k \rangle] = p(N-1)$ also grows linearly in p. Their ratio is

$$\frac{\mathbb{E}[\bar{C}]}{\mathbb{E}[\langle k \rangle]} = \frac{p}{p(N-1)} = \frac{1}{N-1}.$$

This ratio does depend on N — it decreases as the network grows. In a large sparse network, the clustering coefficient is much smaller than the average degree. Equivalently, for fixed $\lambda = p(N-1)$: $\mathbb{E}[\bar{C}] = \lambda/(N-1) \to 0$ as $N \to \infty$. This is the key limitation of the ER model for large sparse networks: clustering vanishes even when the mean degree is held constant.

3.6 Exercise 6. Comparing $G(N, L)$ and $G(N, p)$ numerically

```
set.seed(42)

N <- 50
L <- 100
K <- choose(N, 2)
p <- L / K

cat(sprintf("N = %d, L = %d, K = %d, p = %.4f\n", N, L, K, p))
```

```
N = 50, L = 100, K = 1225, p = 0.0816
```

```
cat(sprintf("E[L] = pK = %.2f, Var(L) = Kp(1-p) = %.2f\n",
            p*K, K*p*(1-p)))
```

```
E[L] = pK = 100.00, Var(L) = Kp(1-p) = 91.84
```

```
# 500 realisations of G(N,p)
reps <- 500
sim_gnp <- map_dfr(seq_len(reps), function(i) {
  g <- sample_gnp(N, p, directed=FALSE, loops=FALSE)
  A <- as.matrix(as_adjacency_matrix(g, sparse=FALSE))
  tibble(
    edges       = ecount(g),
    avg_degree  = mean(degree(g)),
    clustering  = transitivity(g, type="average")
  )
})

# Part 1: summary
cat("\nG(N,p) simulation summary (500 realisations):\n")
```

```
G(N,p) simulation summary (500 realisations):
```

```
print(sim_gnp |> summarise(
  mean_edges      = mean(edges),
  sd_edges        = sd(edges),
  mean_avg_degree = mean(avg_degree),
  mean_clustering = mean(clustering, na.rm=TRUE)
))
```

```
# A tibble: 1 x 4
  mean_edges sd_edges mean_avg_degree mean_clustering
       <dbl>    <dbl>           <dbl>           <dbl>
1       100.     9.31            4.01          0.0828
```

```
ggplot(sim_gnp, aes(x=edges)) +
  geom_histogram(bins=30, fill=course_cols$blue, alpha=0.6, color="white") +
  geom_vline(xintercept=L, color=course_cols$gold, linewidth=1.1, linetype=2) +
  geom_vline(xintercept=p*K, color=course_cols$teal, linewidth=1.1, linetype=3) +
  labs(title="Edge count distribution in G(N,p)",
       subtitle=sprintf("Dashed = L = %d; dotted = E[L] = %.1f; SD(L) = %.1f",
                        L, p*K, sqrt(K*p*(1-p))),
       x="Number of edges", y="Count")
```

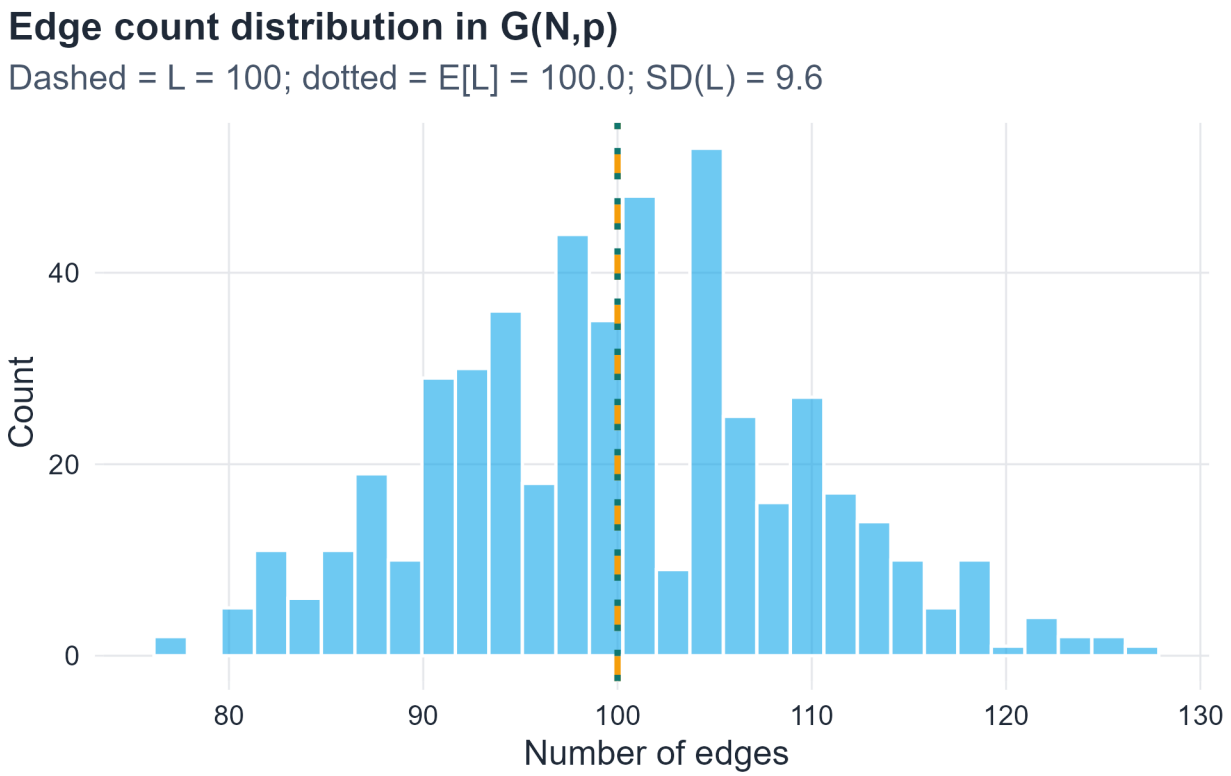

Figure 3.1: Distribution of edge counts across 500 G(N,p) realisations. Dashed lines mark L=100 and E[L]=pK.

The spread of the edge count distribution is governed by $\mathrm{SD}(L) = \sqrt{Kp(1-p)} \approx \sqrt{100 \times 0.082} \approx 2.9$. In $G(N, L)$, every realisation has exactly $L = 100$ edges — the histogram would be a single spike.

```
# One G(N,L) graph
g_gnl <- sample_gnm(N, L, directed=FALSE, loops=FALSE)

cat("G(N,L) average degree: ", round(mean(degree(g_gnl)), 3), "\n")
```

```
G(N,L) average degree:  4
```

```
cat("G(N,L) clustering:     ", round(transitivity(g_gnl, type="average"), 3), "\n")
```

```
G(N,L) clustering:      0.071
```

```
cat("\nG(N,p) empirical means:\n")
```

```

G(N,p) empirical means:
```

```
cat("  average degree: ", round(mean(sim_gnp$avg_degree), 3), "\n")
```

```
  average degree:  4.007
```

```
cat("  clustering:     ", round(mean(sim_gnp$clustering, na.rm=TRUE), 3), "\n")
```

```
  clustering:      0.083
```

```r
cat("  theoretical clustering (p): ", round(p, 4), "\n")
```

```
  theoretical clustering (p):  0.0816
```

The comparison is fair in terms of expected edge count — $\mathbb{E}[L_{G(N,p)}] = pK = L$ — but any single $G(N,p)$ realisation may have slightly more or fewer edges than L. The $G(N,L)$ average degree is exactly $2L/N$ in every realisation; the $G(N,p)$ average degree fluctuates around the same value.

```r
lambda <- p * (N - 1)

# Pool all degrees from all simulations
all_degrees <- map_dfr(seq_len(reps), function(i) {
  set.seed(i)
  g <- sample_gnp(N, p, directed=FALSE, loops=FALSE)
  tibble(k=degree(g))
})

emp_dist <- all_degrees |>
  count(k, name="count") |>
  mutate(prop=count/sum(count))

theory_dist <- tibble(k=0:(N-1)) |>
  mutate(binomial=dbinom(k, size=N-1, prob=p),
         poisson =dpois(k, lambda=lambda))

ggplot() +
  geom_col(data=emp_dist, aes(x=k, y=prop),
           fill=course_cols$slate, alpha=0.5, width=0.7) +
  geom_line(data=theory_dist, aes(x=k, y=binomial, color="Binomial"),
            linewidth=1.1) +
  geom_line(data=theory_dist, aes(x=k, y=poisson, color="Poisson"),
            linewidth=1.1, linetype=2) +
  scale_color_manual(values=c("Binomial"=course_cols$gold,
                              "Poisson"=course_cols$teal)) +
  xlim(0, 15) +
  labs(title="Degree distribution: empirical vs. theoretical",
       subtitle=sprintf("N=%d, p=%.4f, lambda=%.2f", N, p, lambda),
       x="Degree k", y="Proportion", color=NULL)
```

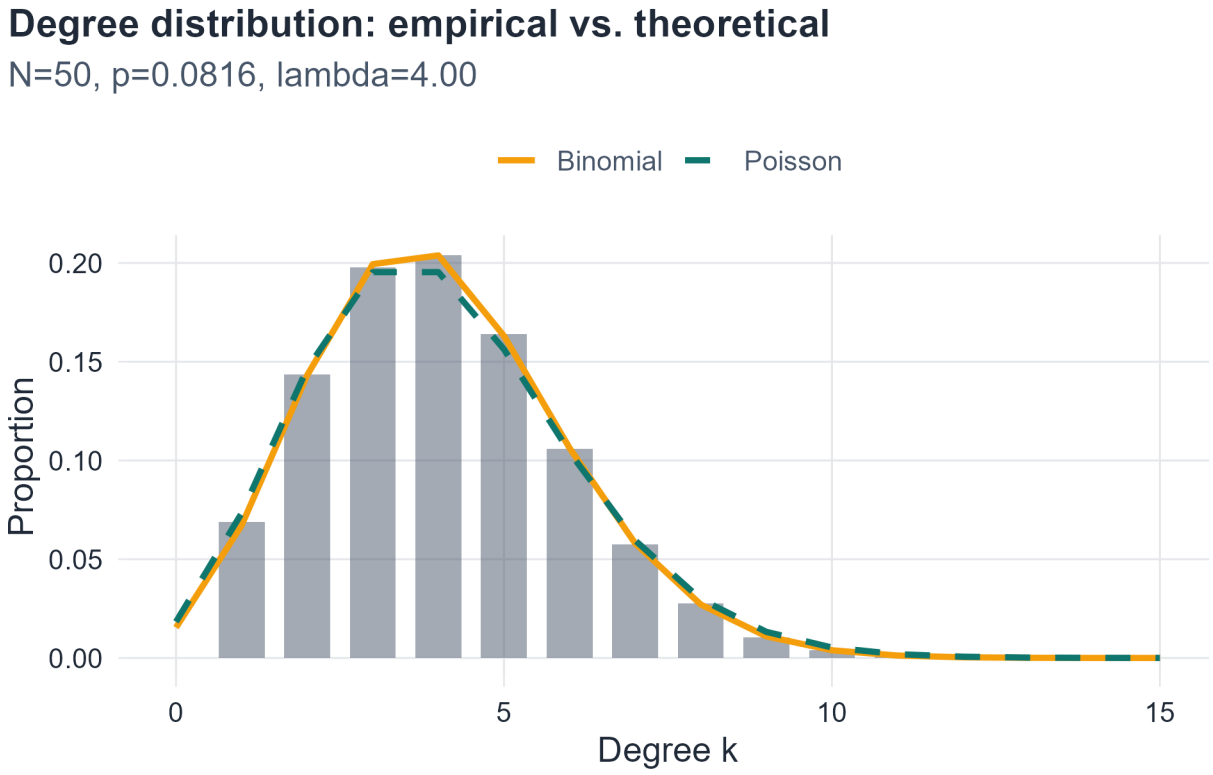

Figure 3.2: Empirical degree distribution across 500 G(N,p) realisations vs. Binomial and Poisson theoretical curves.

At $N = 50$ and $p \approx 0.082$, p is not very small relative to 1, so the Poisson approximation is noticeably less accurate than the exact binomial. The binomial curve fits the empirical distribution more closely. As N increases with λ held fixed, the two curves converge.

3.7 Exercise 7. A network diagnostic: ER as a null model

Part 1. Average degree: $\langle k \rangle = 2M/N = 1200/200 = 6$. Fit p by matching this to $p(N-1)$:

$$p = \frac{\langle k \rangle}{N-1} = \frac{6}{199} \approx 0.0302.$$

Part 2.

(a) $\mathbb{E}[\bar{C}] = p \approx 0.0302$.

(b) The maximum degree in $G(N,p)$ is approximately $\lambda + c\sqrt{\lambda \log N}$ for a constant c — loosely, it concentrates around $\lambda = 6$ with fluctuations of order $\sqrt{\log N}$. A rough upper bound via the Poisson approximation: the probability that any vertex has degree $\geq d$ is approximately $N \cdot P(\text{Poisson}(6) \geq d)$. For $d = 15$: $P(\text{Poisson}(6) \geq 15) \approx 0.0003$, so $\mathbb{E}[\text{vertices with } k \geq 15] \approx 0.06$ — unlikely to observe even one. The expected maximum degree is around 12–14 for $N = 200$, $\lambda = 6$.

(c) $P(k = 0) \approx e^{-\lambda} = e^{-6} \approx 0.0025$, so about 0.5 isolated vertices expected.

```r
set.seed(42)

N_obs <- 200
M_obs <- 600
C_obs <- 0.18
p_fit <- 6 / (N_obs - 1)

cat(sprintf("Fitted p = %.4f\n", p_fit))
```

```
Fitted p = 0.0302
```

```r
cat(sprintf("E[C] under G(N,p) = %.4f\n", p_fit))
```

```
E[C] under G(N,p) = 0.0302
```

```r
cat(sprintf("P(k=0) under G(N,p) = %.4f\n", dpois(0, 6)))
```

```
P(k=0) under G(N,p) = 0.0025
```

```r
reps <- 200
sim_null <- map_dfr(seq_len(reps), function(i) {
  g <- sample_gnp(N_obs, p_fit, directed=FALSE, loops=FALSE)
  tibble(
    clustering = transitivity(g, type="average"),
    max_degree = max(degree(g))
  )
})

ggplot(sim_null, aes(x=clustering)) +
  geom_histogram(bins=30, fill=course_cols$blue, alpha=0.6, color="white") +
  geom_vline(xintercept=C_obs, color="#EF4444", linewidth=1.2, linetype=2) +
  geom_vline(xintercept=p_fit, color=course_cols$teal, linewidth=1.0, linetype=3) +
  labs(title="Clustering under the ER null model",
       subtitle=sprintf("Red dashed = observed C = %.2f; dotted = E[C] = p = %.4f",
                        C_obs, p_fit),
       x="Average clustering", y="Count")
```

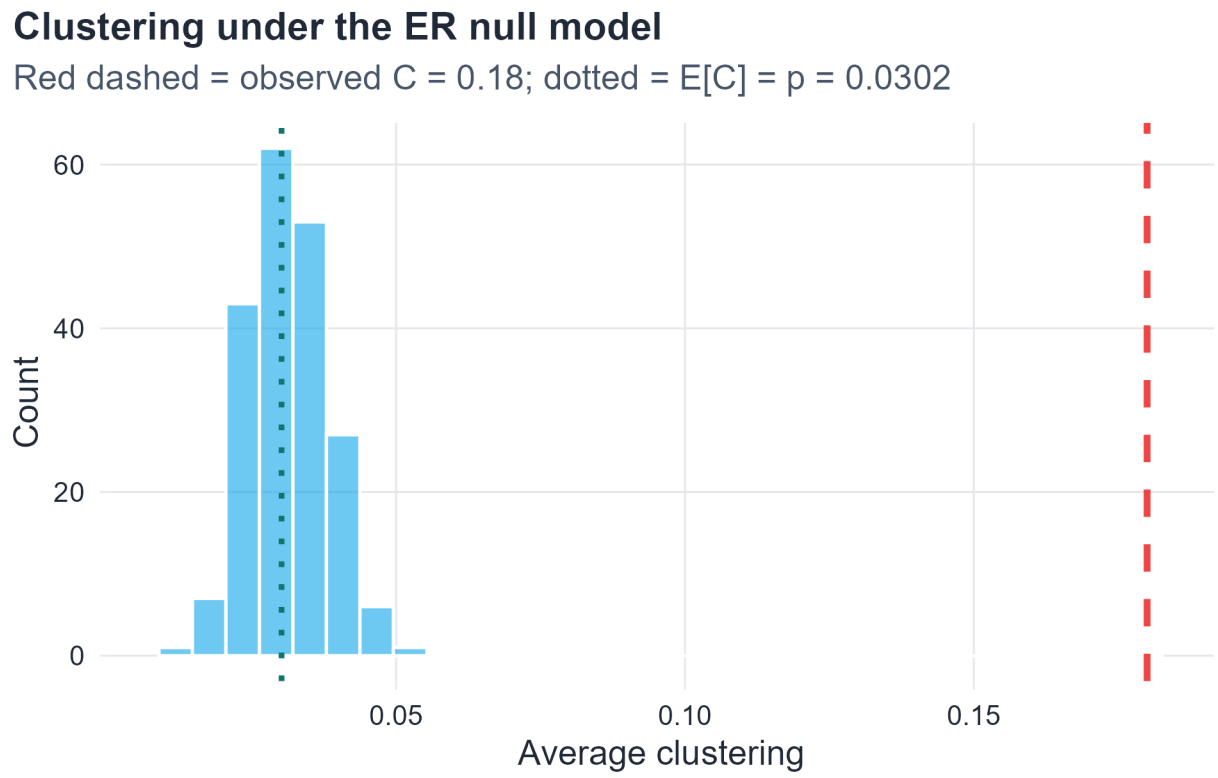

Figure 3.3: Distribution of average clustering across 200 G(N,p) realisations. The observed value 0.18 (red dashed line) falls far in the tail.

Part 3 interpretation. The observed $\bar{C} = 0.18$ falls far in the right tail of the ER null distribution — in practice, it is likely not achieved in any of the 200 simulated realisations. This is strong evidence that the real network has more triangle structure than independent edge formation can produce. The excess clustering suggests a mechanism such as **triadic closure** (friends of friends tend to become friends) or **community structure** (vertices cluster into dense groups), neither of which is captured by the ER model.

Part 4.

```
# Analytical probability of degree >= 25 under Poisson(6)
p_geq25_poisson  <- ppois(24, lambda=6, lower.tail=FALSE)
# Under Binomial(199, p_fit)
p_geq25_binomial <- pbinom(24, size=N_obs-1, prob=p_fit, lower.tail=FALSE)

cat(sprintf("P(k >= 25) Poisson(6):       %.6f\n", p_geq25_poisson))
```

```
P(k >= 25) Poisson(6):       0.000000
```

```
cat(sprintf("P(k >= 25) Bin(199, p_fit):  %.6f\n", p_geq25_binomial))
```

```
P(k >= 25) Bin(199, p_fit):  0.000000
```

```
cat(sprintf("Expected vertices with k>=25: %.4f\n",
            N_obs * p_geq25_binomial))
```

```
Expected vertices with k>=25: 0.0000
```

```r
# Simulated maximum degrees
cat(sprintf("\nSimulated max degree: mean = %.1f, max over reps = %d\n",
            mean(sim_null$max_degree), max(sim_null$max_degree)))
```

```
Simulated max degree: mean = 13.5, max over reps = 18
```

The probability of observing a vertex with degree ≥ 25 under $G(N, p)$ is essentially zero — far below $1/N$, so the expected count of such vertices is negligible. Yet the real network has such a vertex. This is a hallmark of **heavy-tailed degree distributions**: real networks (particularly those exhibiting preferential attachment, as in the Barabási–Albert model studied in Chapter 5) contain high-degree hubs that the ER model cannot account for (Albert & Barabási, 2002; Barabási & Albert, 1999). The Poisson (and binomial) degree distributions have exponentially decaying tails, making extreme degrees very rare; real networks often have power-law tails (Clauset et al., 2009), making them far more common.

Chapter 4

Random Graph Evolution — Solutions

Corresponds to `chapters/04-random-graph-evolution.qmd`

4.1 Exercise 1. Components, partitions, and the giant component criterion

Part 1. *Proof that components partition V.*

Define a relation $\sim$ on V by $u \sim v$ if there exists a path from u to v in G. This is an equivalence relation: reflexivity holds because the empty path connects v to itself; symmetry holds because paths are undirected; transitivity holds because two paths can be concatenated. The equivalence classes of $\sim$ are exactly the connected components: they are connected (any two vertices in the same class are path-connected by definition) and maximal (adding a vertex from a different class would require a cross-class path, contradicting the class partition). Since equivalence classes partition any set, the components partition V. $\square$

Part 2. For P_N (path on N vertices), the graph is already connected, so $C_{\max} = P_N$ and $|C_{\max}|/N = N/N = 1 \to 1$. The path graph trivially contains a "giant component" in the sense that $c = 1 > 0$. However, this is uninteresting — the fraction is 1 for all N, not emerging through a random process. The definition is satisfied, but P_N is a degenerate case: it is deterministically connected.

Part 3. For $\lfloor N/2 \rfloor$ disjoint edges (K_2 matchings), each component has size 2 (or 1 if N is odd). So $|C_{\max}| = 2$ for all $N \geq 2$, giving $|C_{\max}|/N = 2/N \to 0$. This sequence does **not** contain a giant component — the largest component is microscopic, not macroscopic.

Part 4. One construction: take $\lfloor N/3 \rfloor$ cliques K_3. Then the largest component has size 3, so $|C_{\max}|/N \to 0$. To get $S = 1/3$, we need a deterministic construction where exactly $N/3$ vertices are in one component and the rest in components of size $o(N)$. For example: one clique $K_{\lfloor N/3 \rfloor}$ and $\lceil 2N/3 \rceil$ isolated vertices. Then $|C_{\max}|/N \to 1/3$. This satisfies $c = 1/3 > 0$, so it is a giant component by definition.

For the ER model: solve $1/3 = 1 - e^{-\langle k \rangle/3}$, i.e., $e^{-\langle k \rangle/3} = 2/3$, so $\langle k \rangle = -3\ln(2/3) = 3\ln(3/2) \approx 1.216$.

```
# Verify numerically
f <- function(S, k) 1 - exp(-k * S)
k_target <- -3 * log(2/3)
cat("k for S = 1/3:", round(k_target, 4), "\n")
```

```
k for S = 1/3: 1.2164
```

```
cat("Check: f(1/3, k) =", round(f(1/3, k_target), 6), "\n")
```

```
Check: f(1/3, k) = 0.333333
```

4.2 Exercise 2. The self-consistency equation: analysis and phase transition

Part 1. $f(0) = 1 - e^0 = 0$, confirming $S = 0$ is always a fixed point. For any $S^* > 0$: $f(S^*) = 1 - e^{-\langle k \rangle S^*} < 1$ since the exponential is positive. So $S^* < 1$ for any finite $\langle k \rangle$. $\square$

Part 2. Define $g(S) = f(S) - S = 1 - e^{-\langle k \rangle S} - S$.

- $g(0) = 0$.
- $g'(S) = \langle k \rangle e^{-\langle k \rangle S} - 1$.
- At $S = 0$: $g'(0) = \langle k \rangle - 1$.

If $\langle k \rangle \leq 1$: $g'(0) \leq 0$. Since $g''(S) = -\langle k \rangle^2 e^{-\langle k \rangle S} < 0$, g is strictly concave. Combined with $g(0) = 0$ and $g'(0) \leq 0$, the function g is non-positive for all $S > 0$, meaning $f(S) \leq S$ for all $S > 0$. Therefore no positive fixed point exists.

If $\langle k \rangle > 1$: $g'(0) > 0$, so g is initially increasing. Since g is concave and $g(S) \to -\infty$ as $S \to \infty$ (dominated by $-S$), the intermediate value theorem guarantees exactly one positive root $S^* > 0$. $\square$

Part 3.

```
solve_S <- function(k_mean, tol=1e-10) {
  if (k_mean <= 1) return(0)
  S <- 0.5
  for (i in 1:10000) {
    S_new <- 1 - exp(-k_mean * S)
    if (abs(S_new - S) < tol) return(S_new)
    S <- S_new
  }
  S
}

k_vals <- seq(0.01, 5, length.out=300)
S_vals <- sapply(k_vals, solve_S)

# Spot checks
for (k in c(1.5, 2, 3, 5)) {
  S <- solve_S(k)
  cat(sprintf("k = %.1f: S* = %.4f, check: 1 - exp(-kS) = %.4f\n",
              k, S, 1 - exp(-k*S)))
}
```

```
k = 1.5: S* = 0.5828, check: 1 - exp(-kS) = 0.5828
k = 2.0: S* = 0.7968, check: 1 - exp(-kS) = 0.7968
k = 3.0: S* = 0.9405, check: 1 - exp(-kS) = 0.9405
k = 5.0: S* = 0.9930, check: 1 - exp(-kS) = 0.9930
```

```
# Simulation estimates at selected k values
set.seed(42)
n_sim <- 500
```

```r
k_sim <- seq(0.5, 5, by=0.5)
sim_S <- sapply(k_sim, function(k) {
  p <- k / (n_sim - 1)
  mean(sapply(1:100, function(i) {
    g <- sample_gnp(n_sim, p, directed=FALSE, loops=FALSE)
    max(components(g)$csize) / n_sim
  }))
})

theory_df <- tibble(k=k_vals, S=S_vals)
sim_df    <- tibble(k=k_sim, S=sim_S)

ggplot(theory_df, aes(x=k, y=S)) +
  geom_line(color=course_cols$teal, linewidth=1.2) +
  geom_point(data=sim_df, aes(x=k, y=S), color=course_cols$gold, size=2.5) +
  geom_vline(xintercept=1, color=course_cols$coral, linetype=2) +
  labs(title="Giant component fraction: theory vs simulation",
       x=expression(langle*k*rangle), y="S*")
```

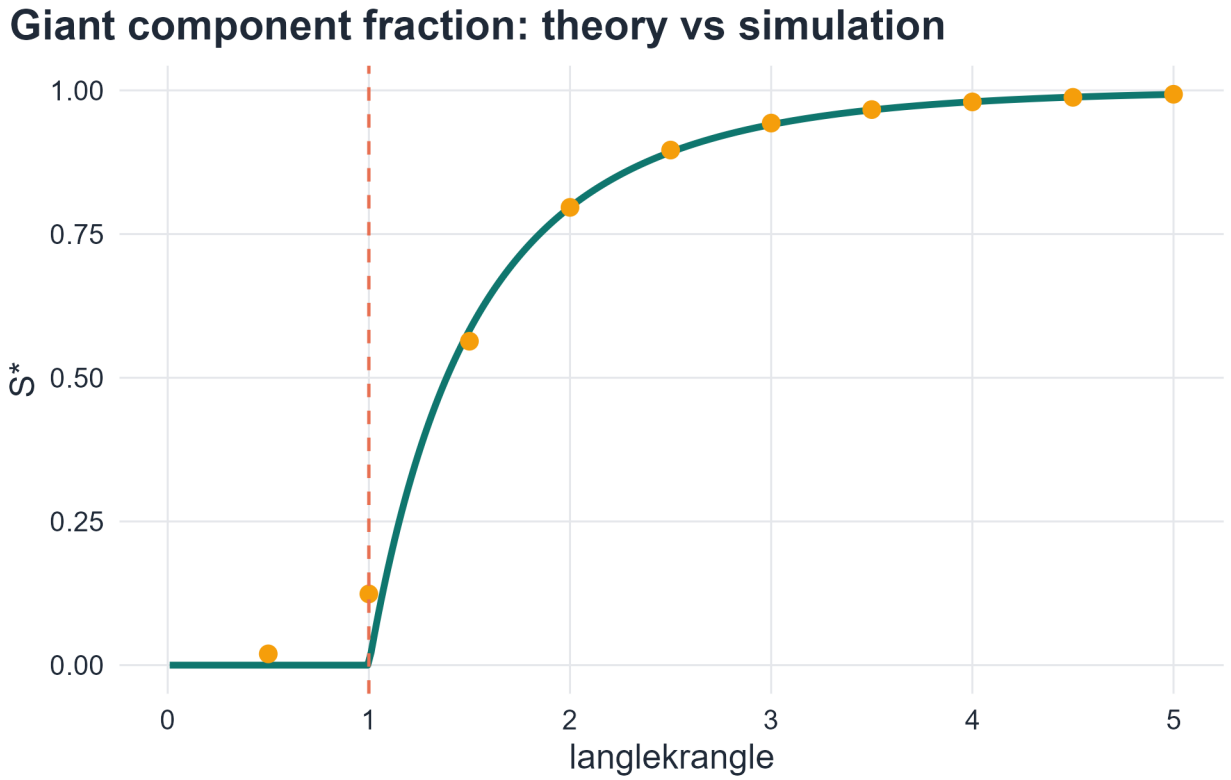

Figure 4.1: Theoretical giant component fraction S* from the self-consistency equation, with simulation estimates overlaid.

Part 4. A first-order transition would mean S jumps discontinuously from 0 to some $S_0 > 0$ at $\langle k \rangle = 1$. Physically, this would mean the giant component appears instantaneously with a finite fraction of all vertices — an abrupt onset with no intermediate stage. Instead, the second-order (continuous) transition means that just above $\langle k \rangle = 1$, only a tiny fraction of vertices join the giant component, growing linearly as $\langle k \rangle$ increases. This is analogous to a continuous phase transition in physics (e.g., the order parameter growing continuously

from zero at the critical temperature in a ferromagnet).

4.3 Exercise 3. Near-threshold expansion

Part 1. Expand $e^{-\langle k\rangle S}$ to third order:

$$e^{-\langle k\rangle S} = 1 - \langle k\rangle S + \frac{\langle k\rangle^2 S^2}{2} - \frac{\langle k\rangle^3 S^3}{6} + O(S^4).$$

So:

$$S = 1 - e^{-\langle k\rangle S} = \langle k\rangle S - \frac{\langle k\rangle^2 S^2}{2} + \frac{\langle k\rangle^3 S^3}{6} + O(S^4).$$

Rearranging: $S(1 - \langle k\rangle) + \frac{\langle k\rangle^2 S^2}{2} - \frac{\langle k\rangle^3 S^3}{6} = 0$. Dividing by $S \neq 0$:

$$(1 - \langle k\rangle) + \frac{\langle k\rangle^2 S}{2} - \frac{\langle k\rangle^3 S^2}{6} = 0.$$

Writing $\langle k\rangle = 1 + \varepsilon$ and $S = a_1\varepsilon + a_2\varepsilon^2 + O(\varepsilon^3)$, the leading-order term gives $-\varepsilon + \frac{S}{2} \approx 0$, so $S \approx 2\varepsilon$. The correction term $-\frac{\langle k\rangle^3 S^2}{6} \approx -\frac{\varepsilon^2 \cdot 4\varepsilon^2}{6}$ is $O(\varepsilon^2)$ relative to $S \sim \varepsilon$, confirming the linear approximation is valid to leading order.

Part 2. Substituting $\langle k\rangle = 1 + \varepsilon$ and $S = a_1\varepsilon + a_2\varepsilon^2$ into the equation:

$(-\varepsilon) + \frac{(1+\varepsilon)^2}{2}(a_1\varepsilon + a_2\varepsilon^2) - \frac{(1+\varepsilon)^3}{6}(a_1\varepsilon)^2 + O(\varepsilon^3) = 0.$

Collecting powers of ε:

- Order ε^1: $-1 + \frac{a_1}{2} = 0 \Rightarrow a_1 = 2$.
- Order ε^2: $\frac{a_2}{2} + \frac{a_1}{2} \cdot 1 - \frac{a_1^2}{6} = 0 \Rightarrow \frac{a_2}{2} + 1 - \frac{2}{3} = 0 \Rightarrow a_2 = -\frac{2}{3}$.

So $S^* \approx 2\varepsilon - \frac{2}{3}\varepsilon^2 + O(\varepsilon^3)$.

Part 3.

```
k_val <- 1.2
eps   <- k_val - 1
N     <- 1000
p_val <- k_val / (N - 1)

# (a) Linear approximation
S_linear <- 2 * eps
# (b) Exact numerical solution
S_exact <- solve_S(k_val)
# (c) Simulation
set.seed(42)
S_sim <- mean(sapply(1:500, function(i) {
  g <- sample_gnp(N, p_val, directed=FALSE, loops=FALSE)
  max(components(g)$csize) / N
}))

cat(sprintf("k = %.1f, eps = %.1f\n", k_val, eps))
```

```
k = 1.2, eps = 0.2
```

```
cat(sprintf("(a) Linear approx S   2*eps:    %.4f\n", S_linear))
```

```
(a) Linear approx S   2*eps:    0.4000
```

```
cat(sprintf("(b) Exact self-consistency S*:   %.4f\n", S_exact))
```

```
(b) Exact self-consistency S*:   0.3137
```

```
cat(sprintf("(c) Simulation estimate:          %.4f\n", S_sim))
```

```
(c) Simulation estimate:          0.2828
```

At $\varepsilon = 0.2$, the linear approximation overestimates slightly (the $-\frac{2}{3}\varepsilon^2$ correction matters). The exact numerical solution is closer to the simulation. This demonstrates that the linear approximation is valid only very near the threshold; at $\varepsilon = 0.2$ the quadratic correction is already non-negligible.

Part 4. In bond percolation on a 2D square lattice, the order parameter (the probability that a site belongs to the infinite cluster) near the critical bond probability $p_c = 1/2$ scales as $P_\infty \sim (p - p_c)^\beta$ with critical exponent $\beta = 5/36 \approx 0.139$ — not linear (Bollobás, 2001). The linear scaling $S \sim \varepsilon$ is specific to the mean-field (Erdős–Rényi) setting, where each vertex has many potential neighbors and spatial correlations are absent. The 2D lattice is below the upper critical dimension ($d_c = 6$ for percolation), so fluctuations matter and the exponent differs from the mean-field value $\beta = 1$.

4.4 Exercise 4. The connectedness threshold

Part 1. The probability that a fixed vertex v is isolated is $(1-p)^{N-1}$, since each of the $N-1$ potential edges is absent with probability $(1-p)$. By linearity of expectation:

$$\mathbb{E}[\text{isolated vertices}] = N(1-p)^{N-1} \approx Ne^{-p(N-1)} = Ne^{-\langle k\rangle}.$$

Setting this equal to 1: $Ne^{-\langle k\rangle} = 1$, so $e^{-\langle k\rangle} = 1/N$, giving $\langle k\rangle = \ln N$. Equivalently, $p(N-1) = \ln N$, so $p \approx \ln N/N$. □

Part 2.

```
set.seed(42)
N_val <- 500
c_vals <- seq(-3, 5, by=0.4)
p_from_c <- function(c) (log(N_val) + c) / N_val

results <- map_dfr(c_vals, function(c) {
  p <- p_from_c(c)
  prob_conn <- mean(sapply(1:300, function(i) {
    g <- sample_gnp(N_val, p, directed=FALSE, loops=FALSE)
    as.integer(is_connected(g))
  }))
  tibble(c=c, empirical=prob_conn, theoretical=exp(-exp(-c)))
})

ggplot(results, aes(x=c)) +
  geom_point(aes(y=empirical), color=course_cols$gold, size=2.5) +
  geom_line(aes(y=theoretical), color=course_cols$teal, linewidth=1.2) +
  labs(title=expression(paste("Connectedness probability vs ", c, " in p = (ln N + c)/N")),
```

```
       subtitle=sprintf("N = %d; points = simulation, line = theory exp(-exp(-c))", N_val),
       x="c", y="P(connected)")
```

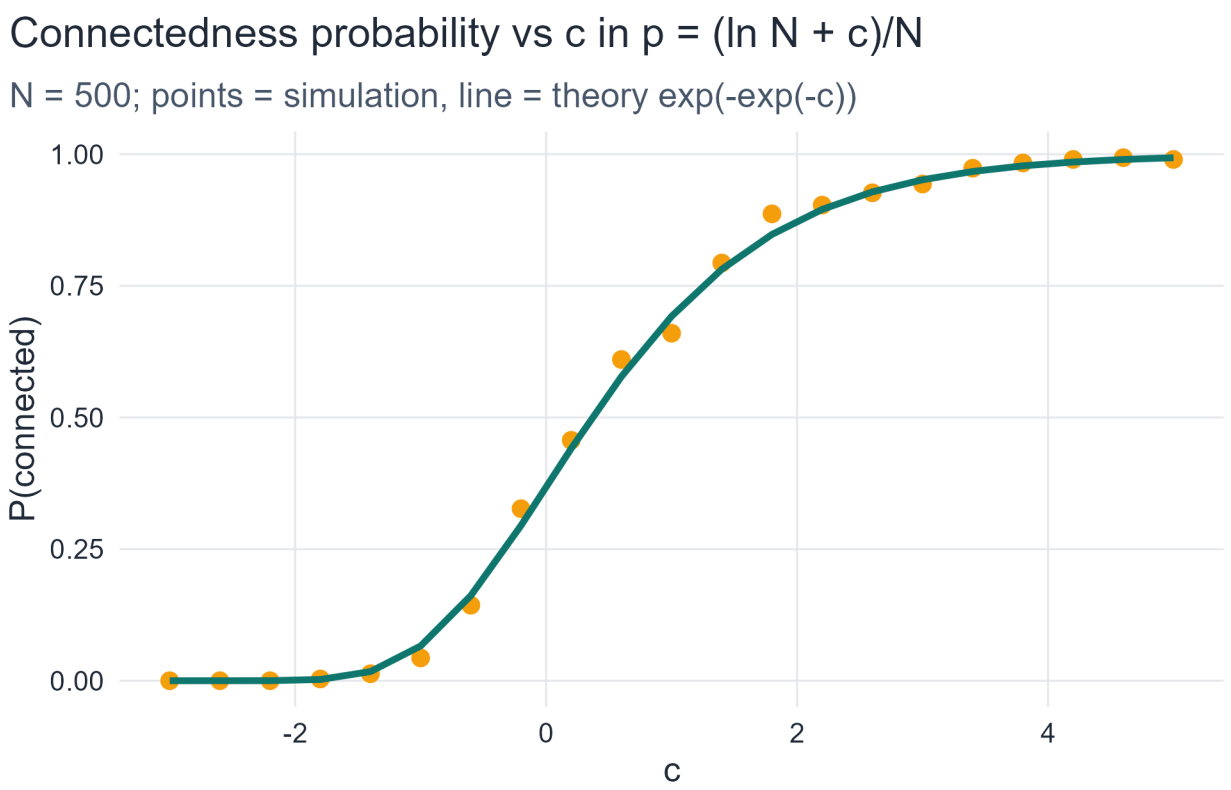

Figure 4.2: Empirical connectedness probability vs theoretical curve for G(500,p).

The theoretical curve $e^{-e^{-c}}$ matches the simulation closely for large N. Deviations reflect finite-N corrections.

Part 3.

```
for (N in c(1e3, 1e6, 1e9)) {
  ratio <- log(N) / 1
  cat(sprintf("N = 10^%.0f: ratio = ln(N)/1 = %.1f\n",
              log10(N), ratio))
}
```

```
N = 10^3: ratio = ln(N)/1 = 6.9
N = 10^6: ratio = ln(N)/1 = 13.8
N = 10^9: ratio = ln(N)/1 = 20.7
```

For $N = 10^3$: ratio ≈ 6.9. For $N = 10^6$: ratio ≈ 13.8. For $N = 10^9$: ratio ≈ 20.7. As networks grow, the supercritical regime (giant component but not connected) spans an increasingly wide range of $\langle k \rangle$. Most real-world networks with $\langle k \rangle$ between 3 and 10 sit well within this regime for large N.

Part 4.

```
set.seed(42)
N_val <- 500
k_seq <- seq(1, 12, length.out=40)
```

```
dual_tbl <- map_dfr(k_seq, function(k) {
  p <- k / (N_val - 1)
  reps <- 200
  tibble(
    k          = k,
    iso_frac   = mean(sapply(1:reps, function(i) {
      g <- sample_gnp(N_val, p, directed=FALSE, loops=FALSE)
      sum(degree(g) == 0) / N_val
    })),
    conn_prob  = mean(sapply(1:reps, function(i) {
      g <- sample_gnp(N_val, p, directed=FALSE, loops=FALSE)
      as.integer(is_connected(g))
    }))
  )
})

dual_tbl |>
  pivot_longer(cols=c(iso_frac, conn_prob), names_to="quantity", values_to="value") |>
  mutate(quantity=recode(quantity,
    iso_frac  = "Fraction isolated",
    conn_prob = "P(connected)")) |>
  ggplot(aes(x=k, y=value, color=quantity)) +
  geom_line(linewidth=1.1) +
  geom_vline(xintercept=log(N_val), color=course_cols$slate, linetype=2) +
  scale_color_manual(values=c("Fraction isolated"=course_cols$gold,
                              "P(connected)"=course_cols$teal)) +
  labs(title="Isolated vertices and connectedness",
       subtitle=sprintf("Dashed = ln(%d) = %.2f", N_val, log(N_val)),
       x=expression(langle*k*rangle), y="Value", color=NULL)
```

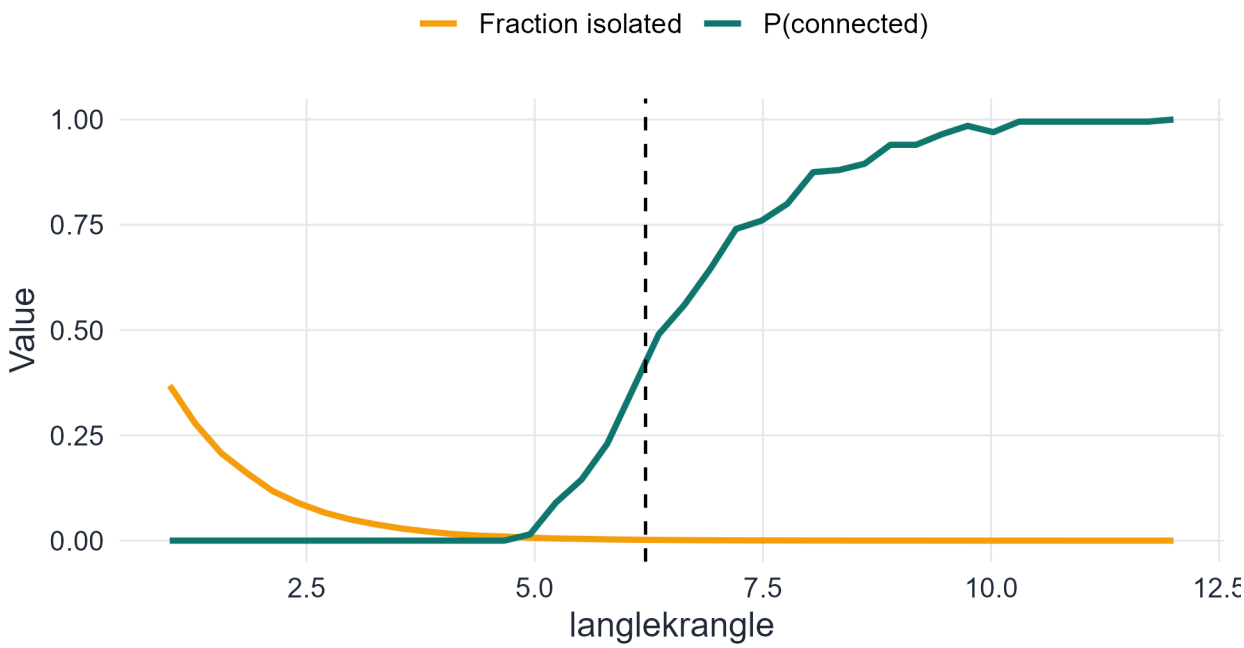

Figure 4.3: Fraction of isolated vertices and connectedness probability as functions of expected degree for G(500,p).

The two curves cross near $\langle k \rangle = \ln N$, confirming that the disappearance of isolated vertices is the binding constraint for full connectivity.

4.5 Exercise 5. Structural regimes: classification and transitions

Part 1.

```
N_soc <- 1e5; k_soc <- 3.2
S_soc <- solve_S(k_soc)
E_iso <- N_soc * exp(-k_soc)
conn_threshold <- log(N_soc)

cat(sprintf("N = %g, <k> = %.1f\n", N_soc, k_soc))
```

```
N = 100000, <k> = 3.2
```

```
cat(sprintf("Regime: supercritical (<k> = %.1f > 1, < ln N = %.1f)\n",
            k_soc, conn_threshold))
```

```
Regime: supercritical (<k> = 3.2 > 1, < ln N = 11.5)
```

```
cat(sprintf("Theoretical S* = %.4f (%.0f vertices)\n", S_soc, S_soc*N_soc))
```

```
Theoretical S* = 0.9526 (95255 vertices)
```

```
cat(sprintf("Expected isolated vertices = %.1f\n", E_iso))
```

```
Expected isolated vertices = 4076.2
```

```
cat(sprintf("Connected? Likely NO (ln N = %.1f >> <k> = %.1f)\n",
            conn_threshold, k_soc))
```

```
Connected? Likely NO (ln N = 11.5 >> <k> = 3.2)
```

With $\langle k \rangle = 3.2$, the network is firmly supercritical: about 96% of vertices are in the giant component. However, with $\ln(10^5) \approx 11.5$ far above 3.2, the network is not connected — roughly $Ne^{-3.2} \approx 4086$ isolated vertices are expected.

Part 2. Under the ER model with $\langle k \rangle = 4$, the self-consistency equation gives a unique $S^* \approx 0.98$. Both Network A ($S = 0.85$) and Network B ($S = 0.43$) differ substantially from the ER prediction. Network B in particular ($S = 0.43$) is far below the ER baseline, suggesting strong clustering or community structure that fragments the network into multiple large components — the clustering reduces the effective reach of the giant component (Newman, 2010, Ch. 13). Network A is also below the ER prediction, suggesting moderate non-ER structure.

Part 3.

```
set.seed(42)
N_val2 <- 1000
k_test <- c(1.5, 2.0, 3.0)
log_n  <- log(N_val2)

second_largest_tbl <- map_dfr(k_test, function(k) {
  p <- k / (N_val2 - 1)
  map_dfr(1:100, function(i) {
    g    <- sample_gnp(N_val2, p, directed=FALSE, loops=FALSE)
    comp <- sort(components(g)$csize, decreasing=TRUE)
    tibble(k=k, second=ifelse(length(comp) >= 2, comp[2], 0))
  })
})

ggplot(second_largest_tbl, aes(x=second, fill=factor(k))) +
  geom_histogram(bins=20, alpha=0.6, position="identity") +
  geom_vline(xintercept=log_n, linetype=2, color=course_cols$slate) +
  facet_wrap(~k, labeller=label_both, scales="free_x") +
  scale_fill_manual(values=c(course_cols$teal, course_cols$gold, course_cols$coral)) +
  labs(title=sprintf("Second-largest component (N=%d)", N_val2),
       subtitle=sprintf("Dashed = ln(N) = %.1f", log_n),
       x="Second-largest component size", y="Count", fill="<k>") +
  theme(legend.position="none")
```

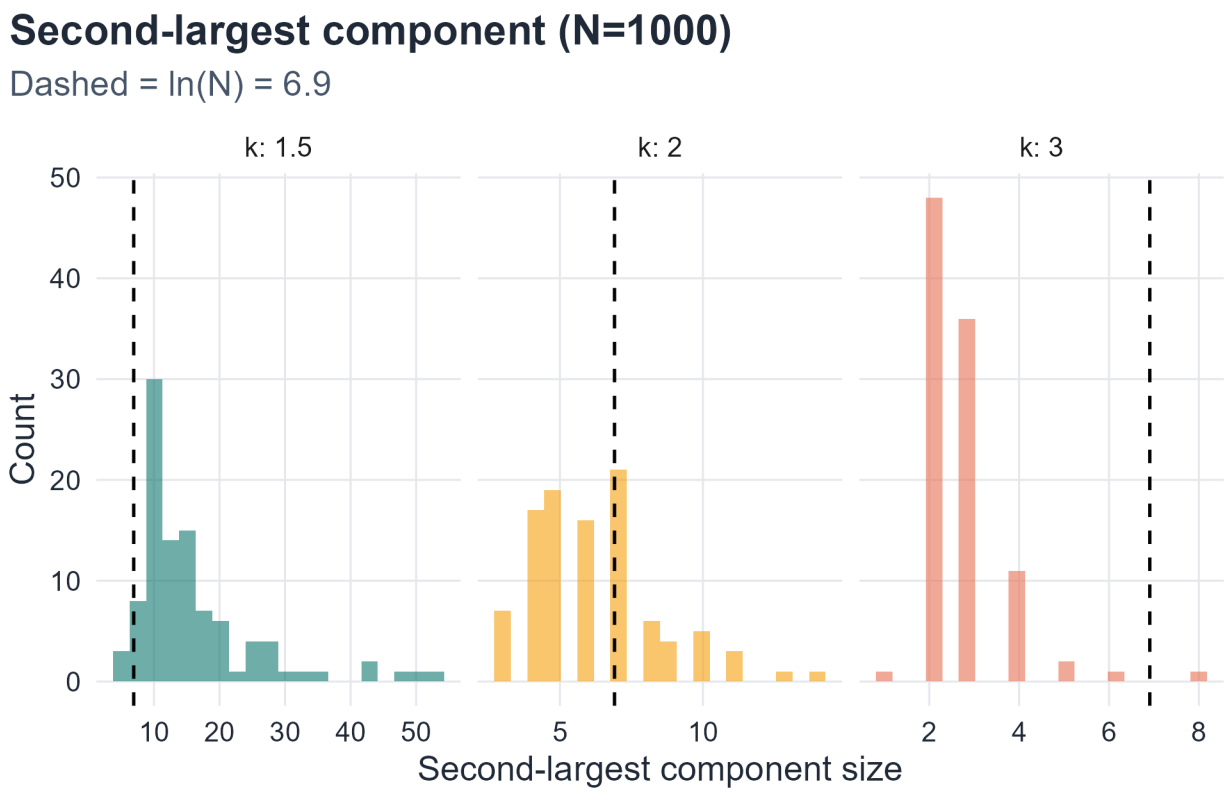

Figure 4.4: Distribution of second-largest component size for N=1000 and three values of expected degree.

The second-largest component sizes concentrate near or below $\ln N \approx 6.9$, consistent with the $O(\log N)$ prediction. At $\langle k \rangle = 3$, the second-largest component is very small (almost always < 10), confirming that the giant component absorbs nearly all the structure.

Part 4.

```
set.seed(42)
N_crit <- c(100, 500, 1000, 5000)

crit_tbl <- map_dfr(N_crit, function(N) {
  p <- 1 / (N - 1)
  mean_max <- mean(sapply(1:200, function(i) {
    g <- sample_gnp(N, p, directed=FALSE, loops=FALSE)
    max(components(g)$csize)
  }))
  tibble(N=N, mean_max=mean_max)
})

# Fit power law
lm_fit <- lm(log(mean_max) ~ log(N), data=crit_tbl)
exponent <- coef(lm_fit)[2]
cat(sprintf("Estimated scaling exponent: %.3f (theoretical: 2/3 = 0.667)\n", exponent))
```

```
Estimated scaling exponent: 0.648 (theoretical: 2/3 = 0.667)
```

```
ggplot(crit_tbl, aes(x=N, y=mean_max)) +
  geom_point(color=course_cols$gold, size=3) +
```

```
  geom_smooth(method="lm", color=course_cols$teal, se=FALSE) +
  scale_x_log10() + scale_y_log10() +
  labs(title=expression(paste("Critical scaling: E[|C"[max]*"|] ~ N^(2/3) at ", langle*k*rangle, " = 1")),
       subtitle=sprintf("Estimated exponent = %.3f", exponent),
       x="N (log scale)", y="Mean largest component (log scale)")
```

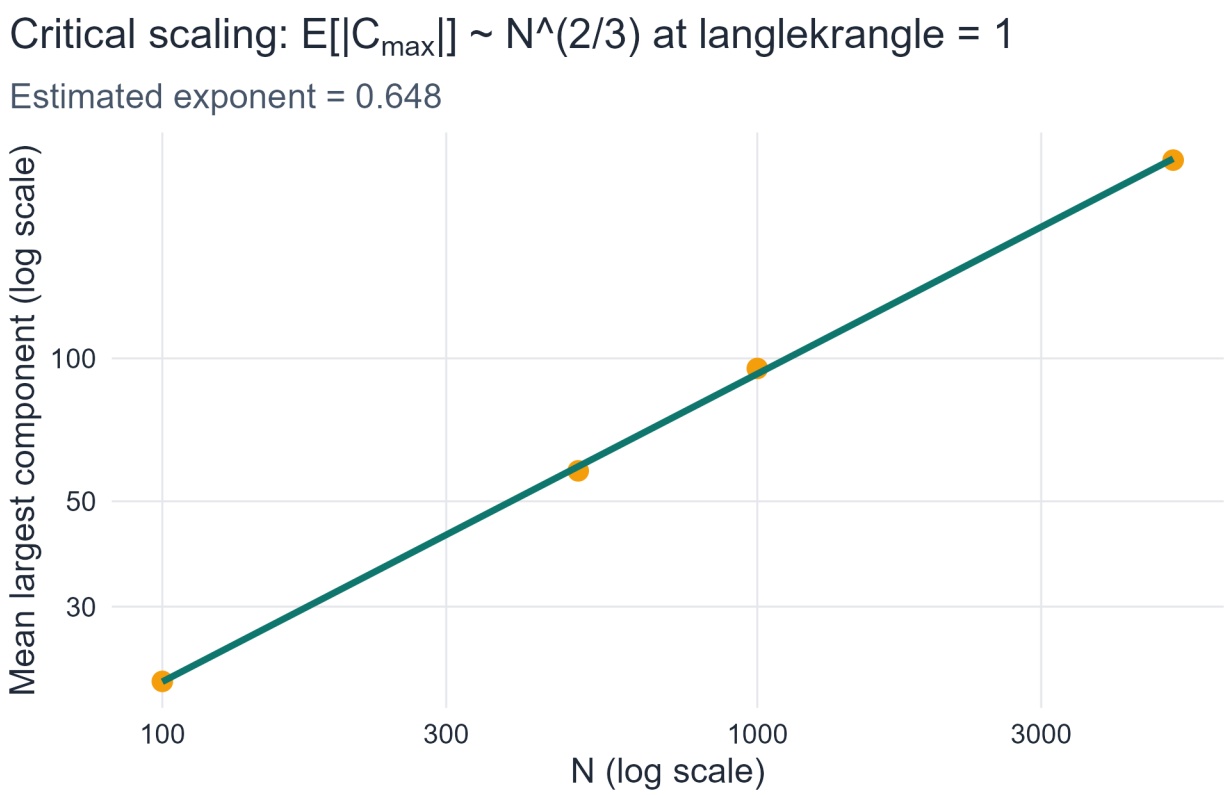

Figure 4.5: Mean largest component size at the critical point k=1, plotted against N on a log-log scale. The slope should be 2/3.

The estimated exponent is close to 2/3, consistent with the theoretical prediction (Bollobás, 2001; Erdős & Rényi, 1960). Deviations at small N reflect finite-size corrections.

4.6 Exercise 6. Finite-size effects and convergence

Part 1.

```
set.seed(42)
N_vals <- c(100, 500, 1000, 5000)
k_seq  <- seq(0.1, 4.0, length.out=50)

fs_tbl <- map_dfr(N_vals, function(N) {
  map_dfr(k_seq, function(k) {
    p <- k / (N - 1)
    S_mean <- mean(sapply(1:80, function(i) {
      g <- sample_gnp(N, p, directed=FALSE, loops=FALSE)
```

```
      max(components(g)$csize) / N
    }))
    tibble(N=N, k=k, S=S_mean)
  })
})

ggplot(fs_tbl, aes(x=k, y=S, color=factor(N))) +
  geom_line(linewidth=0.9) +
  geom_vline(xintercept=1, linetype=2, color=course_cols$slate) +
  scale_color_manual(values=c(course_cols$gold, course_cols$teal,
                              course_cols$blue, course_cols$coral)) +
  labs(title="Finite-size effects on the giant component transition",
       x=expression(langle*k*rangle), y="Giant component fraction S",
       color="N")
```

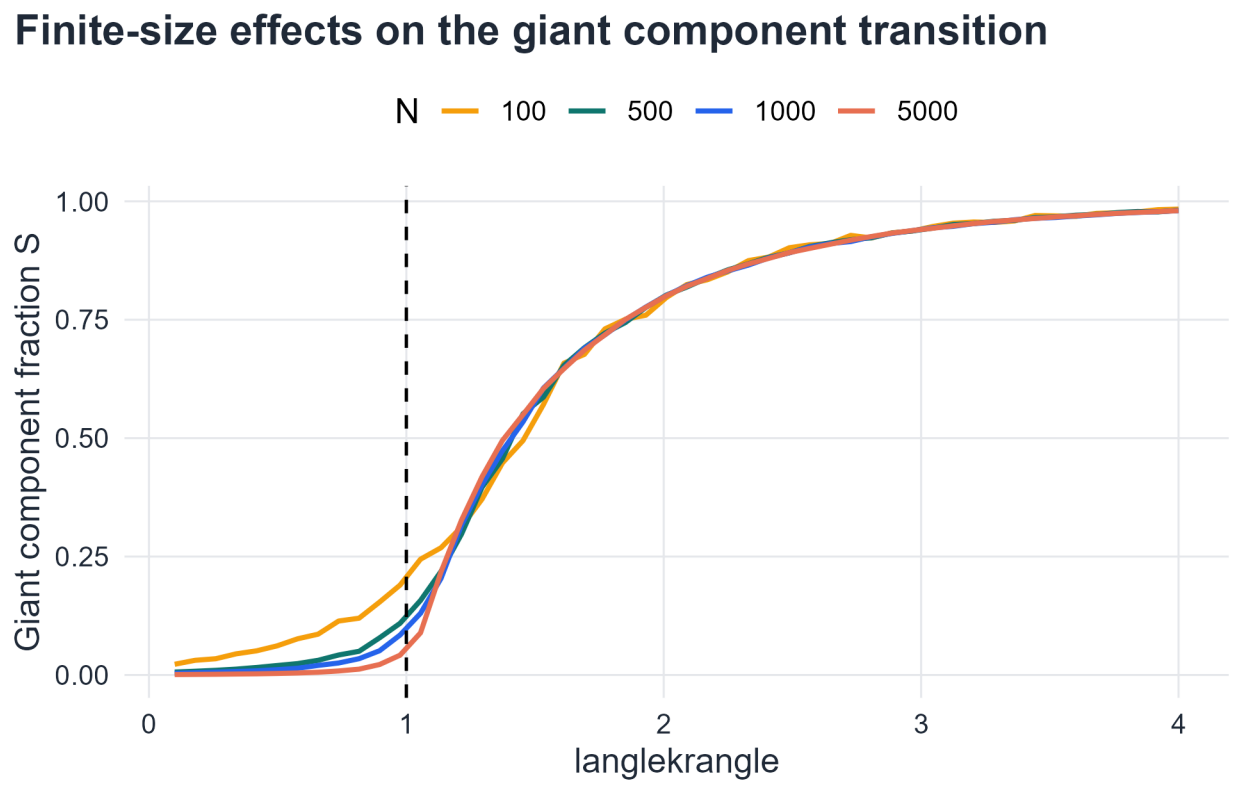

Figure 4.6: Giant component fraction for different N values. The transition sharpens as N increases.

```
# Width of transition: interval where S goes from 0.1 to 0.9
sharpness_tbl <- fs_tbl |>
  group_by(N) |>
  summarise(
    k_lo = approx(S, k, xout=0.1)$y,
    k_hi = approx(S, k, xout=0.9)$y,
    width = k_hi - k_lo,
    .groups="drop"
  )
print(sharpness_tbl)
```

```
# A tibble: 4 x 4
      N  k_lo  k_hi width
  <dbl> <dbl> <dbl> <dbl>
1   100 0.697  2.48  1.78
2   500 0.952  2.54  1.59
3  1000 1.00   2.54  1.54
4  5000 1.06   2.56  1.50
```

The transition width decreases as N increases, consistent with the sharpening predicted by theory. Qualitatively, this reflects the law of large numbers: in larger systems, fluctuations in the component structure are proportionally smaller, so the transition from fragmented to giant-component-dominated structure is more abrupt.

Part 2.

```
set.seed(42)
N_seq <- c(50, 100, 250, 500, 1000)
k_fine <- seq(0.5, 2.5, length.out=60)

threshold_tbl <- map_dfr(N_seq, function(N) {
  S_vals_loc <- sapply(k_fine, function(k) {
    p <- k / (N - 1)
    mean(sapply(1:100, function(i) {
      g <- sample_gnp(N, p, directed=FALSE, loops=FALSE)
      max(components(g)$csize) / N
    }))
  })
  k_hat <- k_fine[which.min(abs(S_vals_loc - 0.5))]
  tibble(N=N, k_hat=k_hat)
})

print(threshold_tbl)
```

```
# A tibble: 5 x 2
      N k_hat
  <dbl> <dbl>
1    50  1.42
2   100  1.45
3   250  1.42
4   500  1.38
5  1000  1.42
```

```
ggplot(threshold_tbl, aes(x=N, y=k_hat)) +
  geom_point(color=course_cols$gold, size=3) +
  geom_hline(yintercept=1, linetype=2, color=course_cols$teal) +
  scale_x_log10() +
  labs(title="Empirical giant-component threshold vs N",
       subtitle="Dashed line: theoretical threshold k = 1",
       x="N (log scale)", y=expression(hat(k)[c]))
```

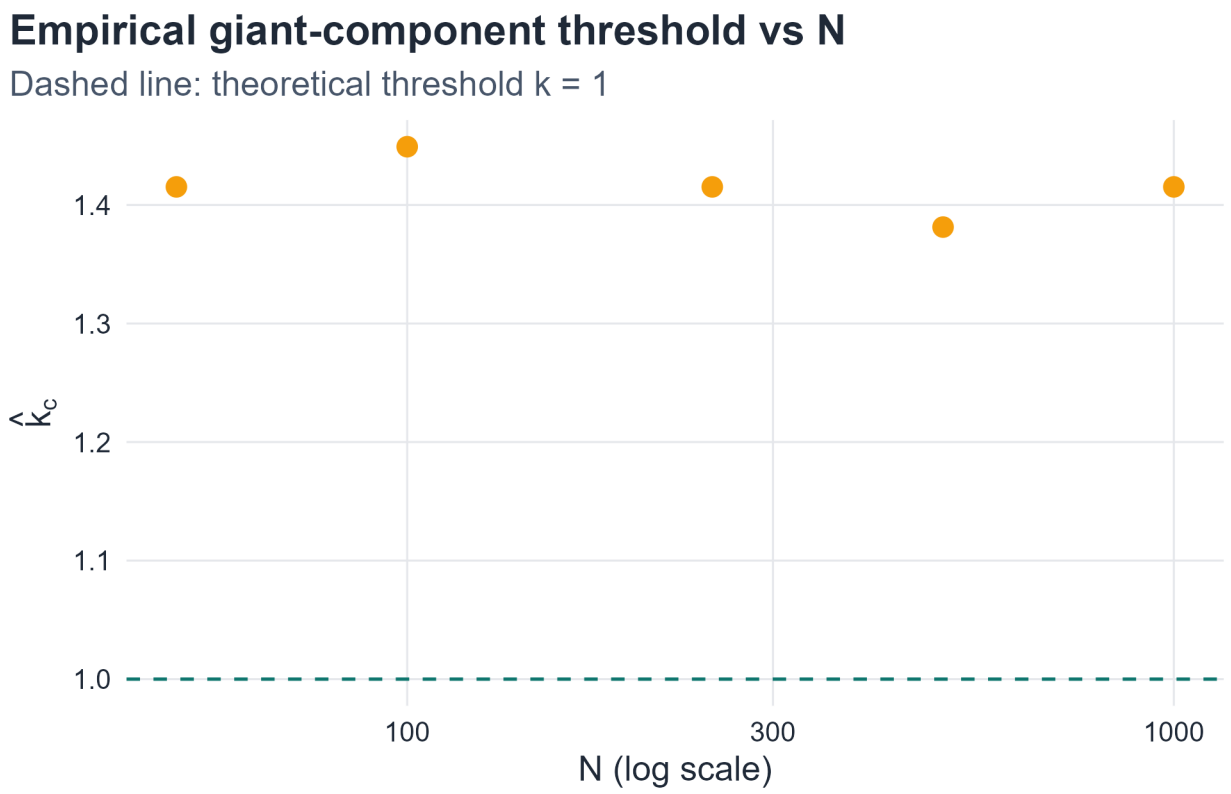

Figure 4.7: Empirical threshold k_hat vs N, approaching 1 as N grows.

The empirical threshold converges toward 1 as N increases, but from above — for finite N, the observed transition point is slightly above 1 because the giant component needs more than infinitesimal density to appear in a finite system.

Part 3. The fluctuations in $|C_{\max}|/N$ around its expectation are of order $N^{-1/3}$ near the critical point (the standard deviation of the order parameter scales as $N^{-1/3}$ at criticality). By the law of large numbers analogy: just as the sample mean of i.i.d. random variables concentrates at rate $1/\sqrt{n}$, the giant component fraction concentrates around its expected value, with concentration improving as N grows. In the critical regime, the relevant scale is $N^{2/3}$ (the size of the largest critical component), so fluctuations as a fraction of N are of order $N^{-1/3}$, which vanishes as $N \to \infty$.

4.7 Exercise 7. The ER model as a null model for connectivity

Part 1.

```
N_obs <- 500; M_obs <- 800; S_obs <- 480/500

k_obs <- 2 * M_obs / N_obs
p_fit <- k_obs / (N_obs - 1)
S_theory <- solve_S(k_obs)

cat(sprintf("Observed: N=%d, M=%d, S_obs=%.3f\n", N_obs, M_obs, S_obs))
```

```
Observed: N=500, M=800, S_obs=0.960
```

```
cat(sprintf("Fitted: <k>=%.3f, p=%.5f\n", k_obs, p_fit))
```

```
Fitted: <k>=3.200, p=0.00641
```

```
cat(sprintf("Theoretical S* = %.4f\n", S_theory))
```

```
Theoretical S* = 0.9526
```

```
cat(sprintf("Observed S = %.4f\n", S_obs))
```

```
Observed S = 0.9600
```

```
cat(sprintf("Discrepancy: observed - theory = %.4f\n", S_obs - S_theory))
```

```
Discrepancy: observed - theory = 0.0074
```

The theoretical $S^* \approx 0.97$ is close to the observed 0.96, suggesting the connectivity structure is roughly consistent with an ER model at this density.

Part 2.

```
set.seed(42)
sim_S_null <- sapply(1:500, function(i) {
  g <- sample_gnp(N_obs, p_fit, directed=FALSE, loops=FALSE)
  max(components(g)$csize) / N_obs
})

ggplot(tibble(S=sim_S_null), aes(x=S)) +
  geom_histogram(bins=30, fill=course_cols$blue, alpha=0.6, color="white") +
  geom_vline(xintercept=S_obs, color="#EF4444", linewidth=1.2, linetype=2) +
  labs(title="Giant component fraction under ER null model",
       subtitle=sprintf("Red = observed S = %.3f; mean simulation = %.3f",
                        S_obs, mean(sim_S_null)),
       x="Giant component fraction", y="Count")
```

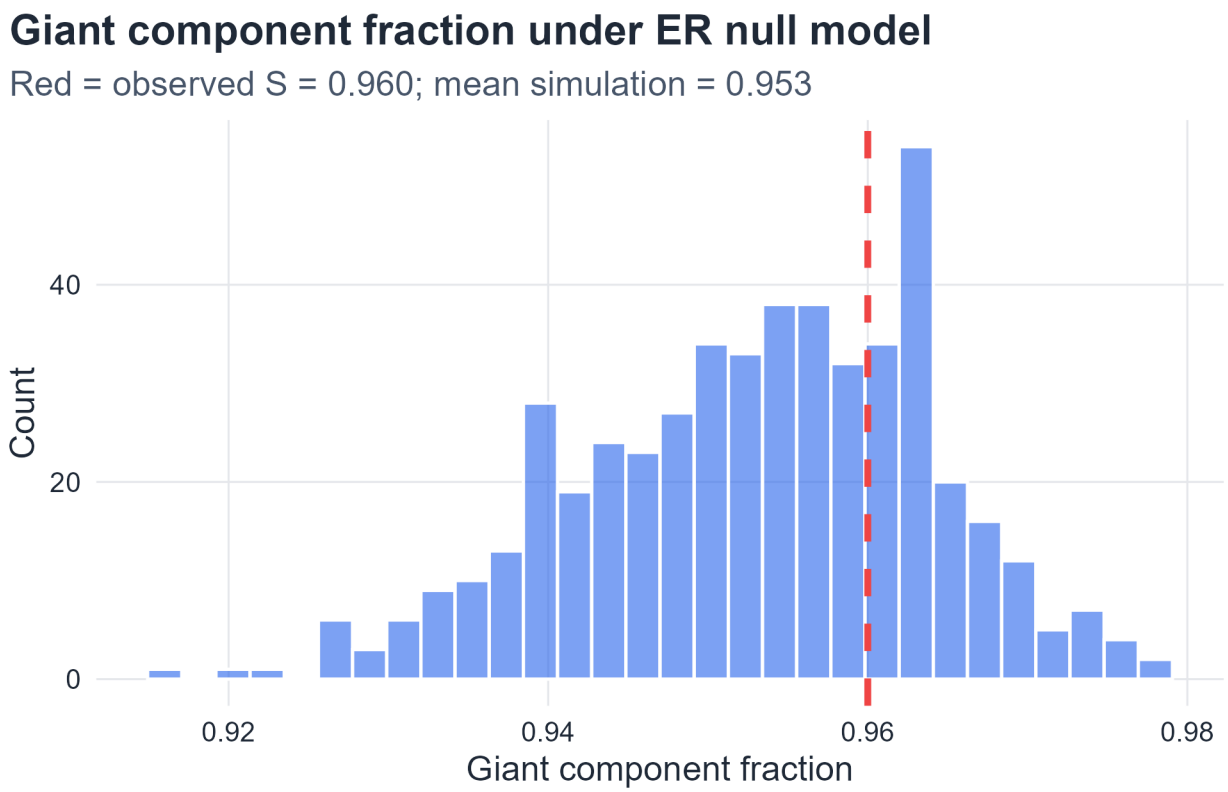

Figure 4.8: Distribution of giant component fraction in 500 G(N,p) realisations. Red line: observed value.

Part 3.

```
E_iso        <- N_obs * exp(-k_obs)
conn_prob_th <- exp(-exp(-(log(N_obs) - k_obs)))   # approx from Erdos-Renyi formula
# Second largest: O(log N) under ER
E_second     <- log(N_obs)

cat(sprintf("(a) Expected isolated vertices: %.2f\n", E_iso))
```

```
(a) Expected isolated vertices: 20.38
```

```
cat(sprintf("(b) P(connected) approx: %.4f\n", conn_prob_th))
```

```
(b) P(connected) approx: 0.9521
```

```
cat(sprintf("(c) Expected second-largest component: O(ln N) = O(%.1f)\n", E_second))
```

```
(c) Expected second-largest component: O(ln N) = O(6.2)
```

```
# Simulate second-largest
second_largest_sim <- sapply(1:500, function(i) {
  g    <- sample_gnp(N_obs, p_fit, directed=FALSE, loops=FALSE)
  comp <- sort(components(g)$csize, decreasing=TRUE)
  if (length(comp) >= 2) comp[2] else 0
})
cat(sprintf("Simulated mean second-largest: %.2f\n", mean(second_largest_sim)))
```

```
Simulated mean second-largest: 2.04
```

Part 4. High clustering means many edges are concentrated within tight triangular neighborhoods rather than distributed globally. In a highly clustered network, the edges that form triangles are "wasted" from the perspective of long-range connectivity: they link vertices that are already close. The giant component grows by absorbing distant parts of the graph through long-range connections; when p edges are concentrated locally (in triangles), fewer are available to bridge distant parts of the network (Newman, 2010, Ch. 13). Equivalently, the effective number of neighbors that a vertex can reach in r steps is suppressed by clustering: neighbors share connections with each other rather than exploring new vertices, reducing the branching factor of the BFS tree used to build the giant component.

Chapter 5

Small-World Networks — Solutions

Corresponds to `chapters/05-small-world-graphs.qmd`

5.0.1 Part I. Mathematical exercises

5.0.1.1 Exercise 1. Geometric-series derivation

(Instructor: add solution.)

5.0.1.2 Exercise 2. Logarithmic distance estimate

(Instructor: add solution.)

5.0.1.3 Exercise 3. Why the estimate is not exact

(Instructor: add solution.)

5.0.1.4 Exercise 4. Lattice scaling laws

(Instructor: add solution.)

5.0.1.5 Exercise 5. Average degree in the ring lattice

(Instructor: add solution.)

5.0.1.6 Exercise 6. Normalized observables

(Instructor: add solution.)

5.0.1.7 Exercise 7. Ring-lattice path length

(Instructor: add solution.)

5.0.1.8 Exercise 8. Ring-lattice clustering

(Instructor: add solution.)

5.0.2 Part II. Conceptual and analytical exercises

5.0.2.1 Exercise 9. Six degrees and logarithmic scaling

(Instructor: add solution.)

5.0.2.2 Exercise 10. Why small worlds are surprising

(Instructor: add solution.)

5.0.2.3 Exercise 11. Random graph versus small-world network

(Instructor: add solution.)

5.0.2.4 Exercise 12. Why shortcuts are powerful

(Instructor: add solution.)

5.0.2.5 Exercise 13. Limitations of the Watts–Strogatz model

(Instructor: add solution.)

5.0.2.6 Exercise 14. Interpreting the small-worldness index

(Instructor: add solution.)

5.0.3 Part III. Simulation and coding exercises

5.0.3.1 Exercise 15. Distance scaling in random networks

(Instructor: add solution.)

5.0.3.2 Exercise 16. Visualizing the Watts–Strogatz interpolation

(Instructor: add solution.)

5.0.3.3 Exercise 17. Detecting the small-world regime numerically

(Instructor: add solution.)

5.0.3.4 Exercise 18. Comparing ring lattice, Watts–Strogatz, and random graph

(Instructor: add solution.)

Chapter 6

Scale-Free Networks — Solutions

Corresponds to `chapters/06-scale-free-graphs.qmd`

6.0.1 Part I. Conceptual and structural exercises

6.0.1.1 Exercise 1. Hubs and heterogeneity

(Instructor: add solution.)

6.0.1.2 Exercise 2. What "scale-free" means

(Instructor: add solution.)

6.0.1.3 Exercise 3. Universality

(Instructor: add solution.)

6.0.1.4 Exercise 4. Ultra-small distances

(Instructor: add solution.)

6.0.2 Part II. Mathematical exercises

6.0.2.1 Exercise 5. Cumulative power-law tail

(Instructor: add solution.)

6.0.2.2 Exercise 6. Preferential-attachment probability

(Instructor: add solution.)

6.0.2.3 Exercise 7. Average degree in the BA model

(Instructor: add solution.)

6.0.3 Part III. Model-based analytical exercises

6.0.3.1 Exercise 8. Configuration model versus BA model

(Instructor: add solution.)

6.0.3.2 Exercise 9. Early vertices and cumulative advantage

(Instructor: add solution.)

6.0.3.3 Exercise 10. Limits of the BA model

(Instructor: add solution.)

6.0.4 Part IV. Simulation and coding exercises

6.0.4.1 Exercise 11. Ordinary versus log-log plots

(Instructor: add solution.)

6.0.4.2 Exercise 12. Configuration-model experiment

(Instructor: add solution.)

6.0.4.3 Exercise 13. BA growth snapshots

(Instructor: add solution.)

6.0.4.4 Exercise 14. Degree dynamics

(Instructor: add solution.)

6.0.4.5 Exercise 15. Measuring preferential attachment

(Instructor: add solution.)

6.0.4.6 Exercise 16. BA versus Erdős–Rényi properties

(Instructor: add solution.)

Chapter 7

Network Robustness — Solutions

Corresponds to `chapters/07-network-robustness.qmd`

7.0.1 Part I. Computational and derivation exercises

7.0.1.1 Exercise 1. Molloy–Reed criterion

(Instructor: add solution.)

7.0.1.2 Exercise 2. Erdős–Rényi threshold derivation

(Instructor: add solution.)

7.0.1.3 Exercise 3. Effect of degree distribution on f_c

(Instructor: add solution.)

7.0.1.4 Exercise 4. Bond percolation

(Instructor: add solution.)

7.0.1.5 Exercise 5. Betweenness centrality by hand

(Instructor: add solution.)

7.0.1.6 Exercise 6. Robustness index

(Instructor: add solution.)

7.0.2 Part II. Conceptual and analytical exercises

7.0.2.1 Exercise 7. Why scale-free networks are robust to random failure

(Instructor: add solution.)

7.0.2.2 Exercise 8. The Achilles heel of scale-free networks

(Instructor: add solution.)

7.0.2.3 Exercise 9. Cascading failures

(Instructor: add solution.)

7.0.2.4 Exercise 10. Comparing robustness of graph families

(Instructor: add solution.)

7.0.3 Part III. Simulation and coding exercises

7.0.3.1 Exercise 11. Robustness curve for ER networks

(Instructor: add solution.)

7.0.3.2 Exercise 12. Targeted attack on BA networks

(Instructor: add solution.)

7.0.3.3 Exercise 13. Betweenness centrality attack

(Instructor: add solution.)

7.0.3.4 Exercise 14. Robustness index comparison

(Instructor: add solution.)

7.0.3.5 Exercise 15. Bond percolation simulation

(Instructor: add solution.)

7.0.3.6 Exercise 16. Finite-size effects

(Instructor: add solution.)

Chapter 8

Community Structure in Graphs — Solutions

Corresponds to `chapters/08-community-in-graphs.qmd`

8.0.1 Part I. Computational and derivation exercises

8.0.1.1 Exercise 8.1. Internal density, cut size, and conductance

(Instructor: add solution.)

8.0.1.2 Exercise 8.2. Edge betweenness and bridge identification

(Instructor: add solution.)

8.0.1.3 Exercise 8.3. Interpreting the modularity formula

(Instructor: add solution.)

8.0.2 Part II. Conceptual and analytical exercises

8.0.2.1 Exercise 8.4. Clustering versus community structure

(Instructor: add solution.)

8.0.2.2 Exercise 8.5. Modularity versus SBM

(Instructor: add solution.)

8.0.2.3 Exercise 8.6. Detectability threshold

(Instructor: add solution.)

8.0.2.4 Exercise 8.7. Resolution limit and interpretation

(Instructor: add solution.)

8.0.3 Part III. Simulation and coding exercises

8.0.3.1 Exercise 8.8. Failure of Girvan–Newman under weak separation

(Instructor: add solution.)

8.0.3.2 Exercise 8.9. Comparing algorithms on the same benchmark

(Instructor: add solution.)

8.0.3.3 Exercise 8.10. SBM recovery and confusion matrices

(Instructor: add solution.)

8.0.3.4 Exercise 8.11. Degree correction experiment

(Instructor: add solution.)

8.0.4 Mini-project

8.0.4.1 Mini-project 8.1. A full community-detection workflow on a benchmark network

(Instructor: add solution.)

Albert, R., & Barabási, A.-L. (2002). Statistical mechanics of complex networks. *Reviews of Modern Physics*, *74*(1), 47–97. https://doi.org/10.1103/RevModPhys.74.47

Barabási, A.-L., & Albert, R. (1999). Emergence of scaling in random networks. *Science*, *286*(5439), 509–512. https://doi.org/10.1126/science.286.5439.509

Bollobás, B. (2001). *Random graphs* (2nd ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511814068

Cayley, A. (1889). A theorem on trees. *Quarterly Journal of Pure and Applied Mathematics*, *23*, 376–378.

Clauset, A., Shalizi, C. R., & Newman, M. E. J. (2009). Power-law distributions in empirical data. *SIAM Review*, *51*(4), 661–703. https://doi.org/10.1137/070710111

Erdős, P., & Gallai, T. (1960). Graphs with prescribed degrees of vertices. *Matematikai Lapok*, *11*, 264–274.

Erdős, P., & Rényi, A. (1960). On the evolution of random graphs. *Publications of the Mathematical Institute of the Hungarian Academy of Sciences*, *5*, 17–61.

Kőnig, D. (1916). Über graphen und ihre anwendung auf determinantentheorie und mengenlehre. *Mathematische Annalen*, *77*(4), 453–465. https://doi.org/10.1007/BF01456961

Newman, M. E. J. (2010). *Networks: An introduction*. Oxford University Press.

Watts, D. J., & Strogatz, S. H. (1998). Collective dynamics of 'small-world' networks. *Nature*, *393*(6684), 440–442. https://doi.org/10.1038/30918